

Project 2: Social Network Mining

ECE 232E Summer 2026
Prof. Vwani Roychowdhury
UCLA, Dept. of ECE

Posted: Sunday, June 12, 2026
Due: Sunday, July 26, 2026 at 11:59 PM PST

By Enoch Yeo 906630606 / Nicolas Jeong Lee 605309755 / James Chen 005399315

1. Facebook Network

1. Structural Properties of the Facebook Network

Question 1

```
In [1]: import igraph as ig
import numpy as np
import matplotlib.pyplot as plt
import random
import math

g = ig.Graph.Read_Edgelist("facebook_combined.txt", directed=False)
```

Question 1.1

```
In [2]: print("Number of Nodes = ", g.vcount())
print("Number of Edges = ", g.ecount())
```

Number of Nodes = 4039
Number of Edges = 88234

Question 1.2

```
In [3]: print("Connected = ", g.is_connected())
```

Connected = True

Question 2

```
In [4]: print("Diameter = ", g.diameter())
```

Diameter = 8

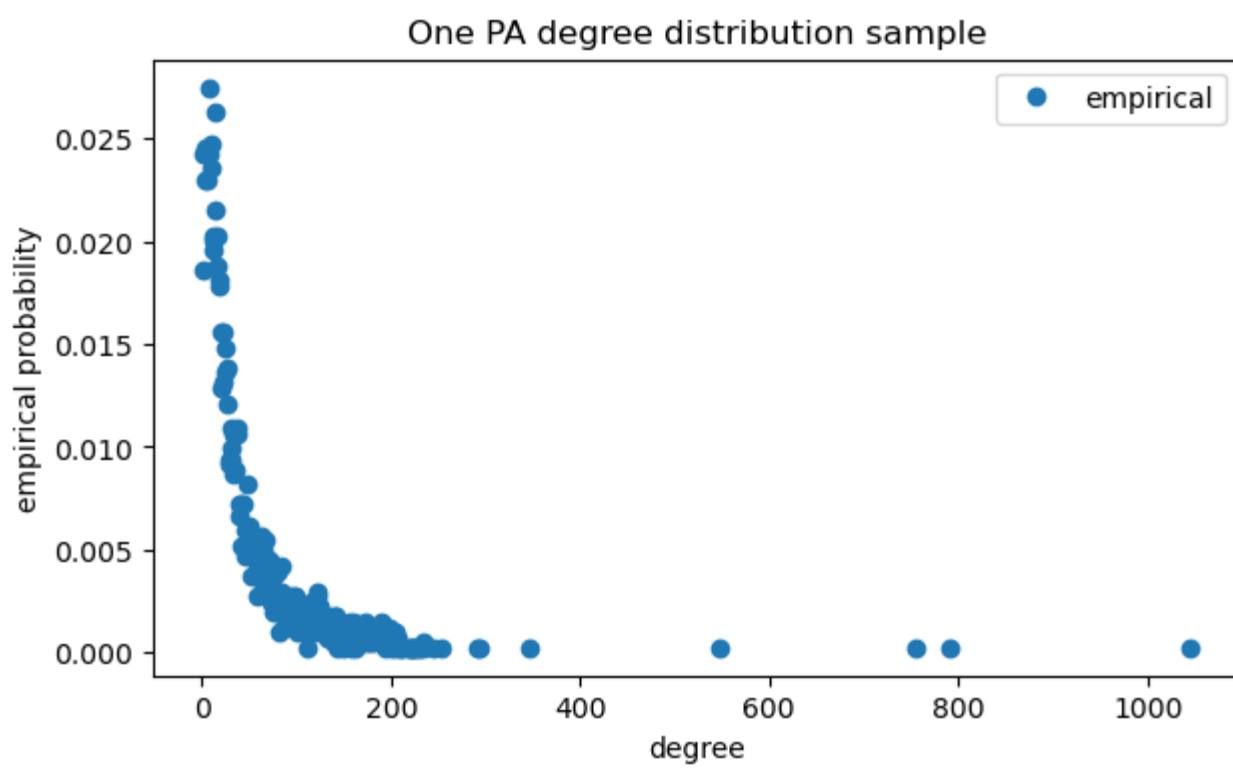
Question 3

```
In [5]: deg = np.asarray(g.degree())
print("Average Degree = ", sum(deg)/g.vcount())
k, counts = np.unique(deg, return_counts=True)
prob = counts / counts.sum()

mask = (k >= 1) & (counts > 0)
slope, intercept = np.polyfit(np.log(k[mask]), np.log(prob[mask]), 1)
print("rough log-log slope:", slope)

plt.figure(figsize=(7, 4))
plt.plot(k, prob, "o", label="empirical")
plt.legend()
plt.xlabel("degree")
plt.ylabel("empirical probability")
plt.title("One PA degree distribution sample")
plt.show()
```

Average Degree = 43.69101262688784
rough log-log slope: -1.1801644192508813



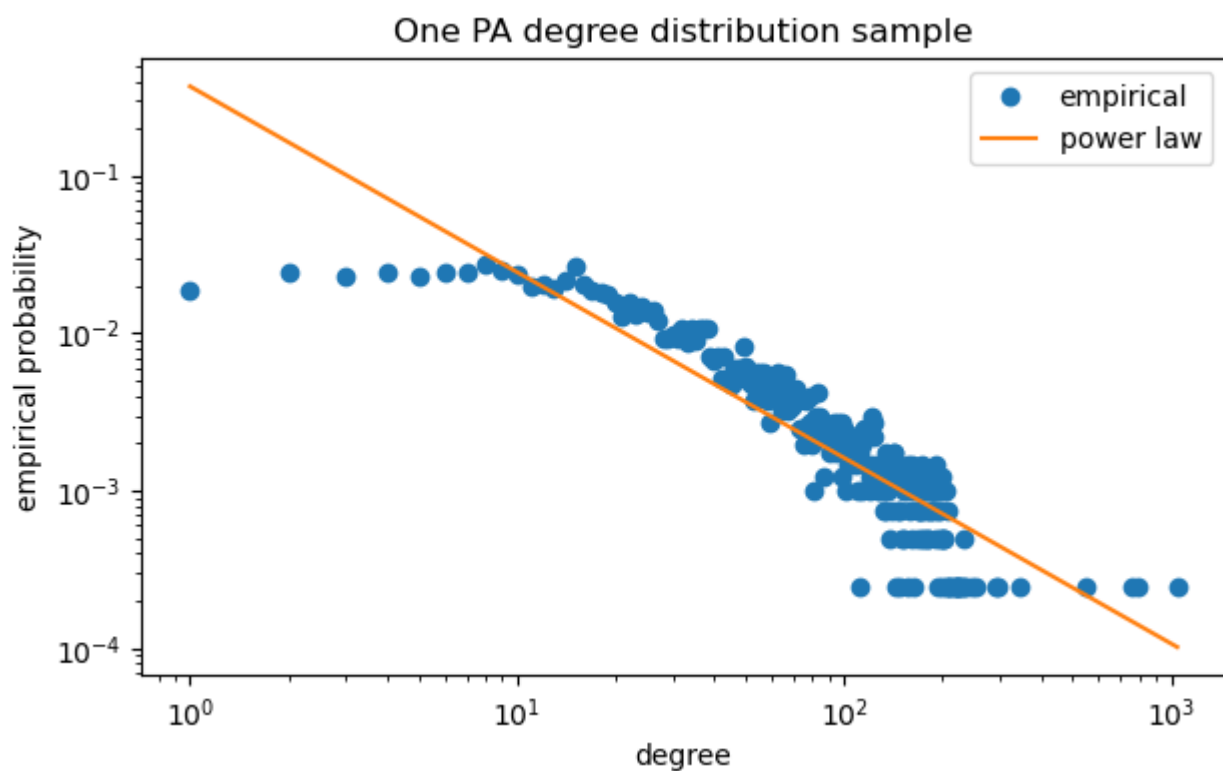
Question 4

```
In [6]: deg = np.asarray(g.degree())
k, counts = np.unique(deg, return_counts=True)
prob = counts / counts.sum()

mask = (k >= 1) & (counts > 0)
slope, intercept = np.polyfit(np.log(k[mask]), np.log(prob[mask]), 1)
print("rough log-log slope:", slope)

plt.figure(figsize=(7, 4))
plt.loglog(k, prob, "o", label="empirical")
plt.loglog(k, np.exp(intercept) * k**slope, label="power law")
plt.legend()
plt.xlabel("degree")
plt.ylabel("empirical probability")
plt.title("One PA degree distribution sample")
plt.show()
```

rough log-log slope: -1.1801644192508813



2. Personalized Network

Question 5

```
In [7]: nodes = [0] + g.neighbors(0)
pn = g.induced_subgraph(nodes)
print("Number of Nodes = ", pn.vcount())
print("Number of Edges = ", pn.ecount())
```

Number of Nodes = 348
Number of Edges = 2866

Question 6

```
In [8]: print("Diameter = ", pn.diameter())
```

Diameter = 2

Lower Bound: Every node in the Personalized Network is connected to v_i , so the distance between a node u and v_i is 1. Likewise, if two neighbors of v_i are also connected, the distance between them is 1. Therefore, the diameter is at least 1.

Upper Bound: Since v_i is connected to every neighbor, the distance between two of those neighbors is 2 since it has to go through v_i . Therefore, the diameter

is at most 2.

Question 7

For the lower bound, it means that every single node are connected to each other and not just the node v_i . So the neighbors of node v_i are also connected to each other. As a result, the distance between any node is 1. This is a fully connected network.

For the upper bound, there is at least one pair of neighbors of v_i that are not connected to each other. So the network is not fully connected. Therefore, there are two nodes that, to get the distance between them, have to go through v_i .

3. Core node's personalized network

Question 8

```
In [9]: count = 0
core_node = []
for node in range(0, g.vcount()):
    if len(g.neighbors(node)) > 200:
        core_node.append(node)
        count += 1
print("Core Nodes = ", core_node)
print("Num of Core Nodes = ", count)

total_degree = 0
for node in core_node:
    total_degree += g.degree(node)
print("Average Degree of Core Nodes = ", total_degree / len(core_node))
```

Core Nodes = [0, 107, 348, 483, 1086, 1199, 1352, 1431, 1584, 1589, 1663, 1684, 1730, 1746, 1768, 1800, 1827, 1888, 1912, 1941, 1985, 1993, 2047, 2078, 2123, 2142, 2206, 2218, 2229, 2233, 2240, 2266, 2347, 2410, 2464, 2507, 2543, 2560, 2611, 3437]

Num of Core Nodes = 40

Average Degree of Core Nodes = 279.375

3.1 Core node's personalized network

Question 9

```
In [10]: def get_communities(g):
    results = {}

    # Fast-Greedy
    fg = g.community_fastgreedy()
    results["Fast-Greedy"] = fg.as_clustering()

    # Edge-Betweenness
    eb = g.community_edge_betweenness()
    results["Edge-Betweenness"] = eb.as_clustering()

    # Infomap
    results["Infomap"] = g.community_infomap()

    return results

def draw_network(pn, clustering, layout, ax, title):
    pal = ig.RainbowPalette(n=max(len(clustering), 2))
    ig.plot(
        clustering,
        target=ax,
        layout=layout,
        palette=pal,
        vertex_size=10,
        vertex_frame_width=0.3,
        edge_color="lightgray",
        edge_width=0.4
    )
    ax.set_title(title)
    ax.set_axis_off()

def plot_node(node_id):
    pn = g.induced_subgraph([node_id] + g.neighbors(node_id))
    layout = pn.layout("fr")
    results = get_communities(pn)

    print(f"Core Node ID {node_id + 1}")
    fig, axes = plt.subplots(1, 3, figsize=(15, 5))
    for ax, (name, cl) in zip(axes, results.items()):
        print(f"{name}: {len(cl)} Communities, Modularity = {cl.modularity:.4f}")
        draw_network(pn, cl, layout, ax, f"{name}\nModularity = {cl.modularity:.4f}")
    fig.suptitle(f"Core Node ID {node_id + 1}", fontsize=14)
    plt.tight_layout()
    plt.show()
```

```
In [11]: plot_node(0)
```

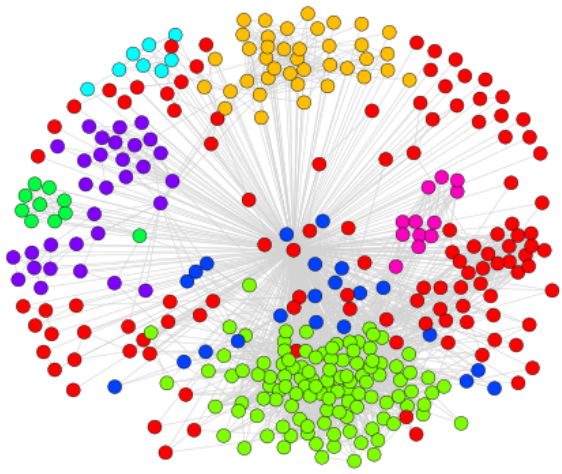
Core Node ID 1

Fast-Greedy: 8 Communities, Modularity = 0.4131

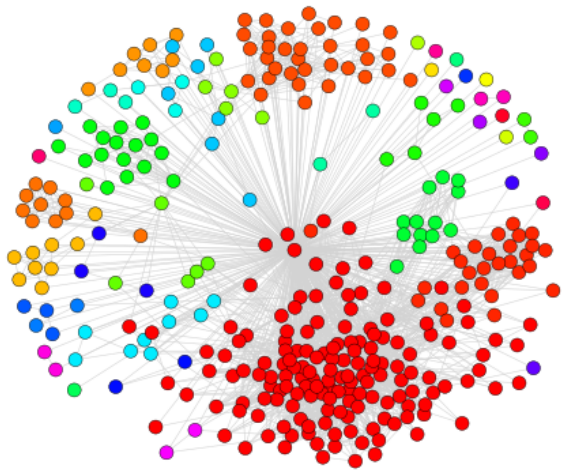
Edge-Betweenness: 41 Communities, Modularity = 0.3533

Infomap: 24 Communities, Modularity = 0.3892

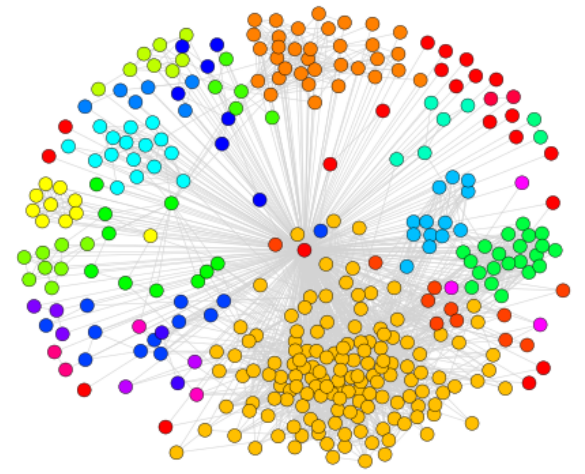
Fast-Greedy
Modularity = 0.4131



Core Node ID 1
Edge-Betweenness
Modularity = 0.3533



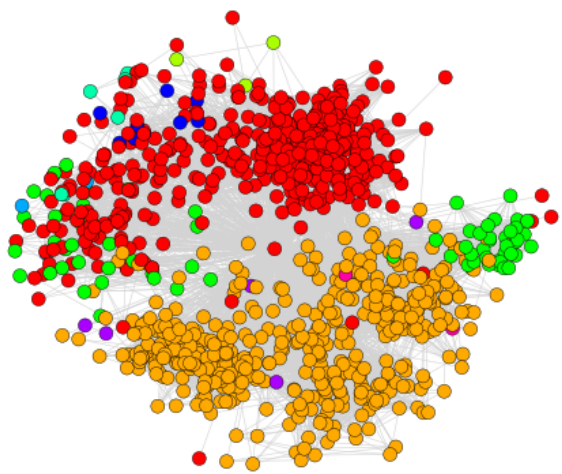
Infomap
Modularity = 0.3892



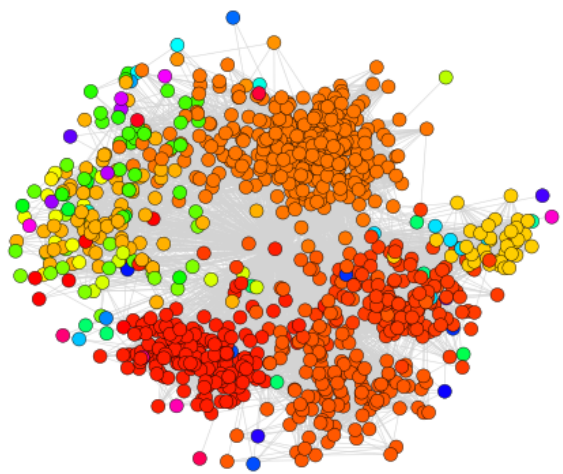
In [12]: `plot_node(107)`

Core Node ID 108
Fast-Greedy: 9 Communities, Modularity = 0.4359
Edge-Betweenness: 52 Communities, Modularity = 0.5068
Infomap: 22 Communities, Modularity = 0.5085

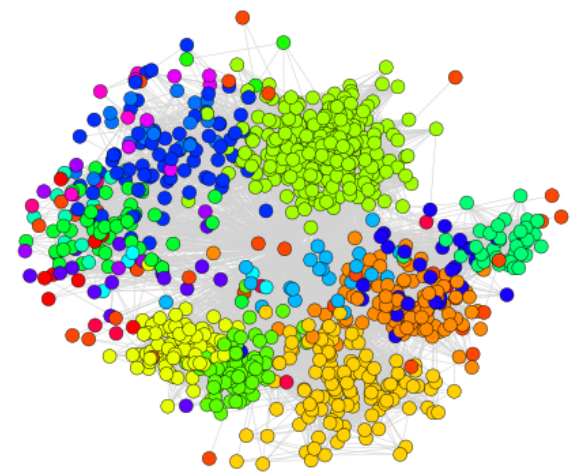
Fast-Greedy
Modularity = 0.4359



Core Node ID 108
Edge-Betweenness
Modularity = 0.5068



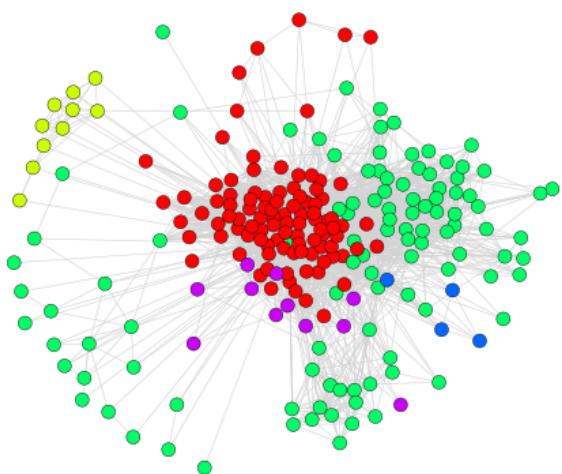
Infomap
Modularity = 0.5085



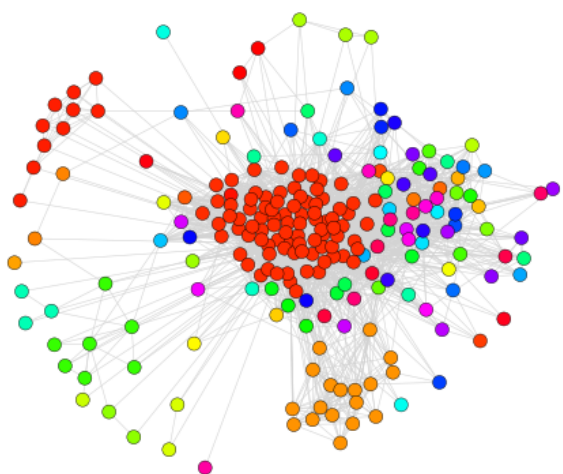
In [13]: `plot_node(348)`

Core Node ID 349
Fast-Greedy: 5 Communities, Modularity = 0.2517
Edge-Betweenness: 104 Communities, Modularity = 0.1335
Infomap: 9 Communities, Modularity = 0.2168

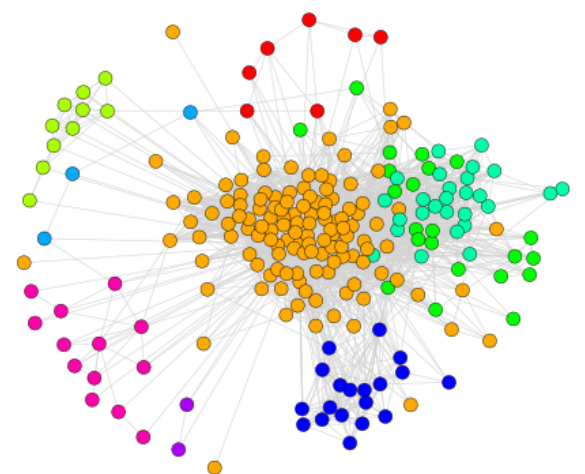
Fast-Greedy
Modularity = 0.2517



Core Node ID 349
Edge-Betweenness
Modularity = 0.1335



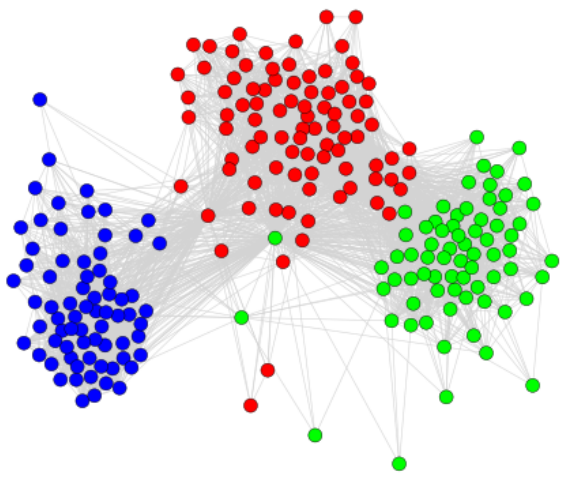
Infomap
Modularity = 0.2168



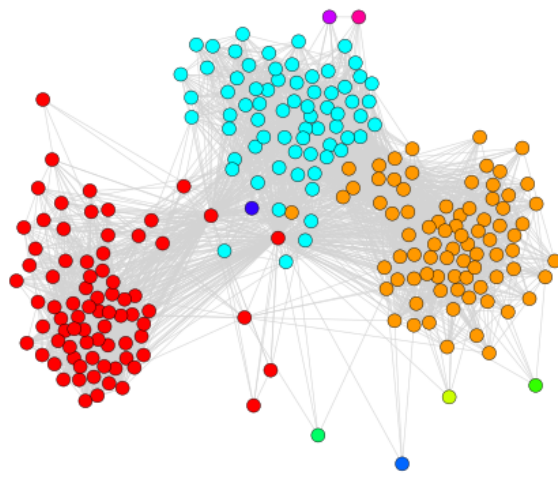
In [14]: `plot_node(483)`

Core Node ID 484
Fast-Greedy: 3 Communities, Modularity = 0.5070
Edge-Betweenness: 10 Communities, Modularity = 0.4891
Infomap: 4 Communities, Modularity = 0.5153

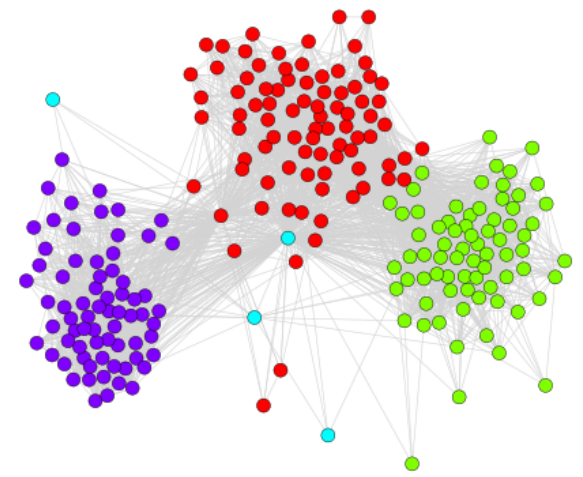
Fast-Greedy
Modularity = 0.5070



Core Node ID 484
Edge-Betweenness
Modularity = 0.4891



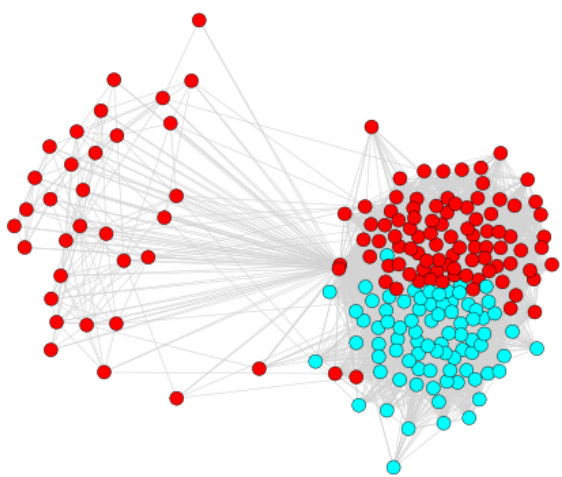
Infomap
Modularity = 0.5153



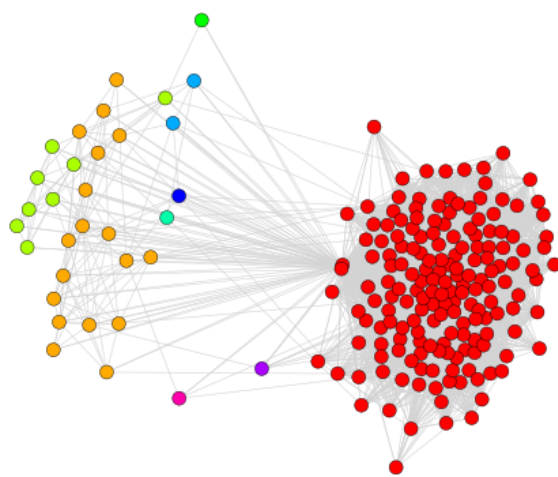
In [15]: `plot_node(1086)`

Core Node ID 1087
Fast-Greedy: 2 Communities, Modularity = 0.1455
Edge-Betweenness: 9 Communities, Modularity = 0.0276
Infomap: 4 Communities, Modularity = 0.0269

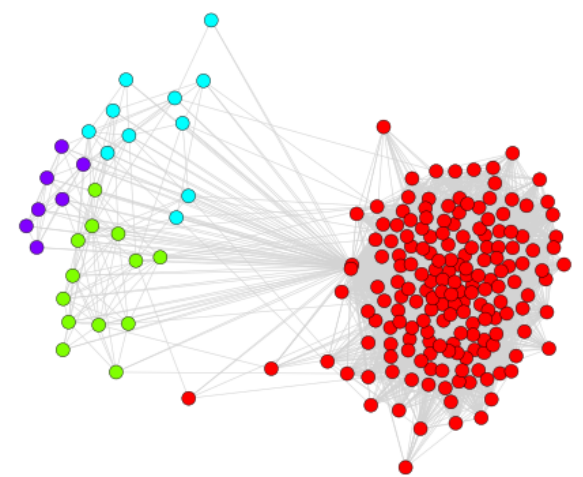
Fast-Greedy
Modularity = 0.1455



Core Node ID 1087
Edge-Betweenness
Modularity = 0.0276



Infomap
Modularity = 0.0269



3.2 Community Structure with the core node removed

Question 10

```
In [16]: def plot_node_v2(node_id):
    pn = g.induced_subgraph([node_id] + g.neighbors(node_id))
    pn.delete_vertices(0)
    layout = pn.layout("fr").coords

    results = get_communities(pn)

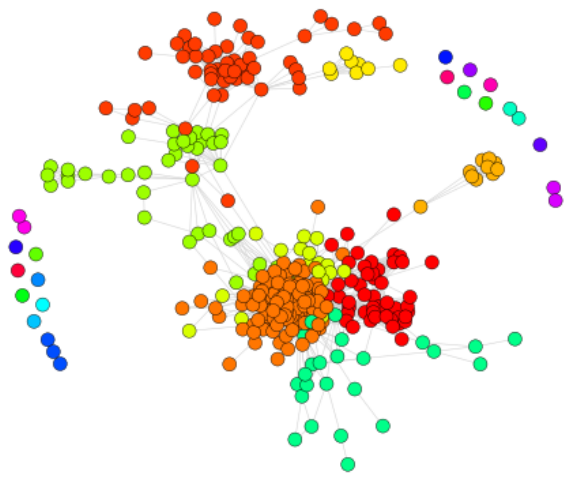
    print(f"Core Node ID {node_id + 1} Deleted")
    fig, axes = plt.subplots(1, 3, figsize=(15, 5))
    for ax, (name, cl) in zip(axes, results.items()):
        print(f"{name}: {len(cl)} Communities, Modularity = {cl.modularity:.4f}")
        draw_network(pn, cl, layout, ax, f"{name}\nModularity = {cl.modularity:.4f}")
    fig.suptitle(f"Core Node ID {node_id + 1} Deleted Modified Personalized Network", fontsize=14)
    plt.tight_layout()
    plt.show()
```

In [17]: `plot_node_v2(0)`

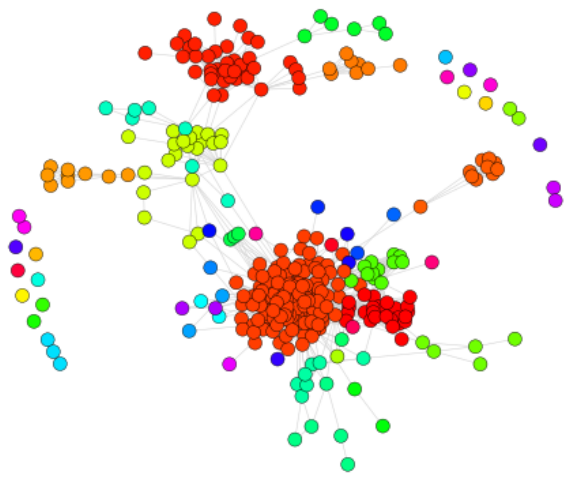
Core Node ID 1 Deleted
Fast-Greedy: 26 Communities, Modularity = 0.4419
Edge-Betweenness: 50 Communities, Modularity = 0.4161
Infomap: 36 Communities, Modularity = 0.4192

Core Node ID 1 Deleted Modified Personalized Network

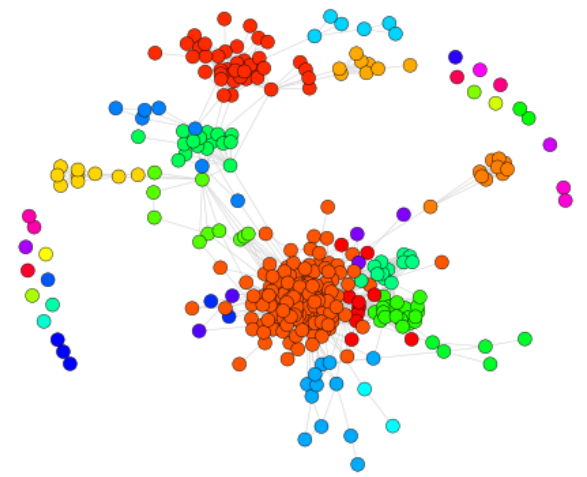
Fast-Greedy
Modularity = 0.4419



Edge-Betweenness
Modularity = 0.4161



Infomap
Modularity = 0.4192



In [18]: `plot_node_v2(107)`

Core Node ID 108 Deleted

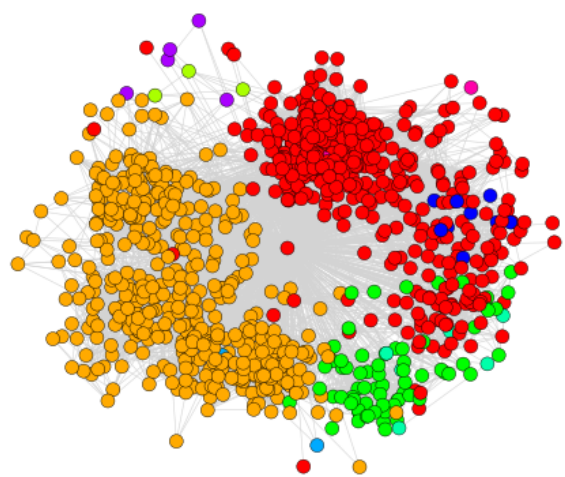
Fast-Greedy: 9 Communities, Modularity = 0.4360

Edge-Betweenness: 52 Communities, Modularity = 0.5056

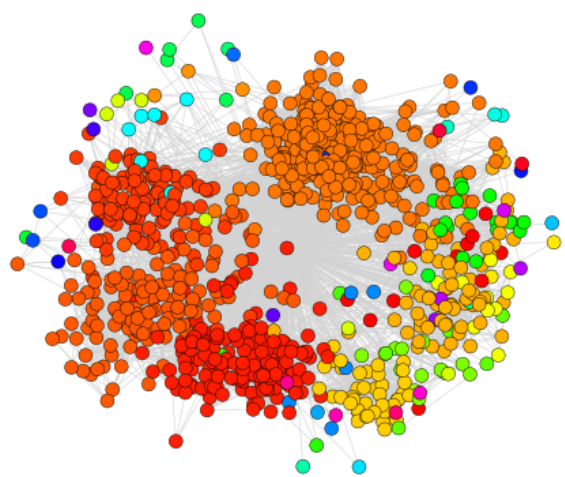
Infomap: 21 Communities, Modularity = 0.5093

Core Node ID 108 Deleted Modified Personalized Network

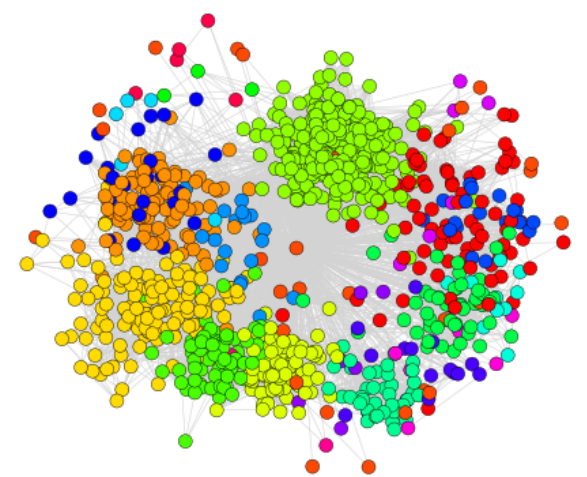
Fast-Greedy
Modularity = 0.4360



Edge-Betweenness
Modularity = 0.5056



Infomap
Modularity = 0.5093



In [19]: `plot_node_v2(348)`

Core Node ID 349 Deleted

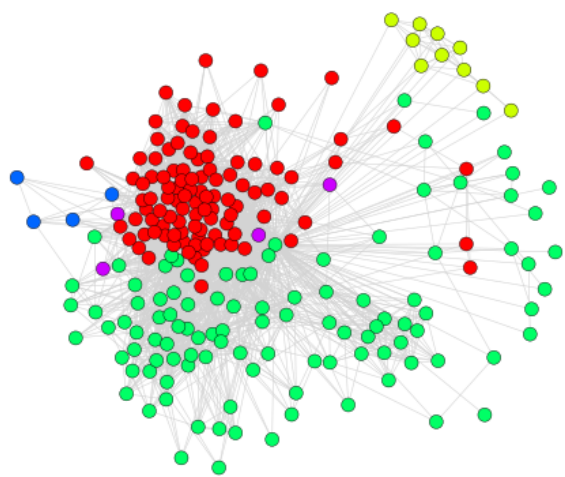
Fast-Greedy: 5 Communities, Modularity = 0.2502

Edge-Betweenness: 103 Communities, Modularity = 0.1337

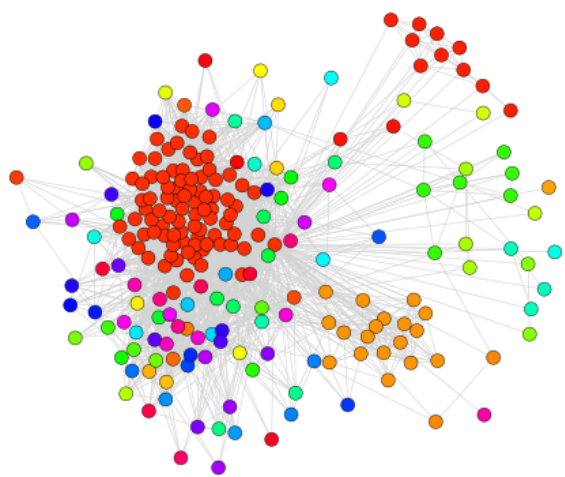
Infomap: 9 Communities, Modularity = 0.2171

Core Node ID 349 Deleted Modified Personalized Network

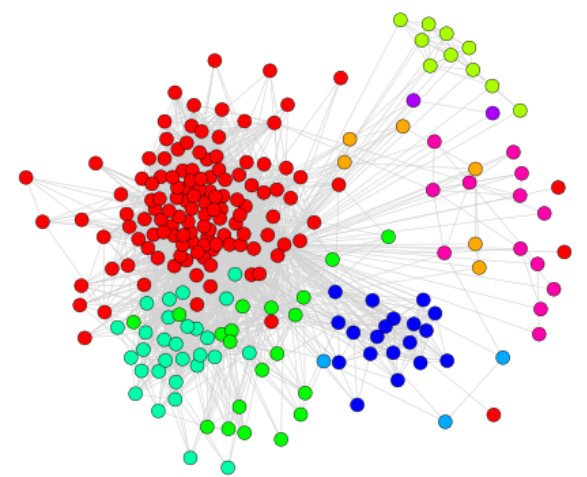
Fast-Greedy
Modularity = 0.2502



Edge-Betweenness
Modularity = 0.1337



Infomap
Modularity = 0.2171



In [20]: `plot_node_v2(483)`

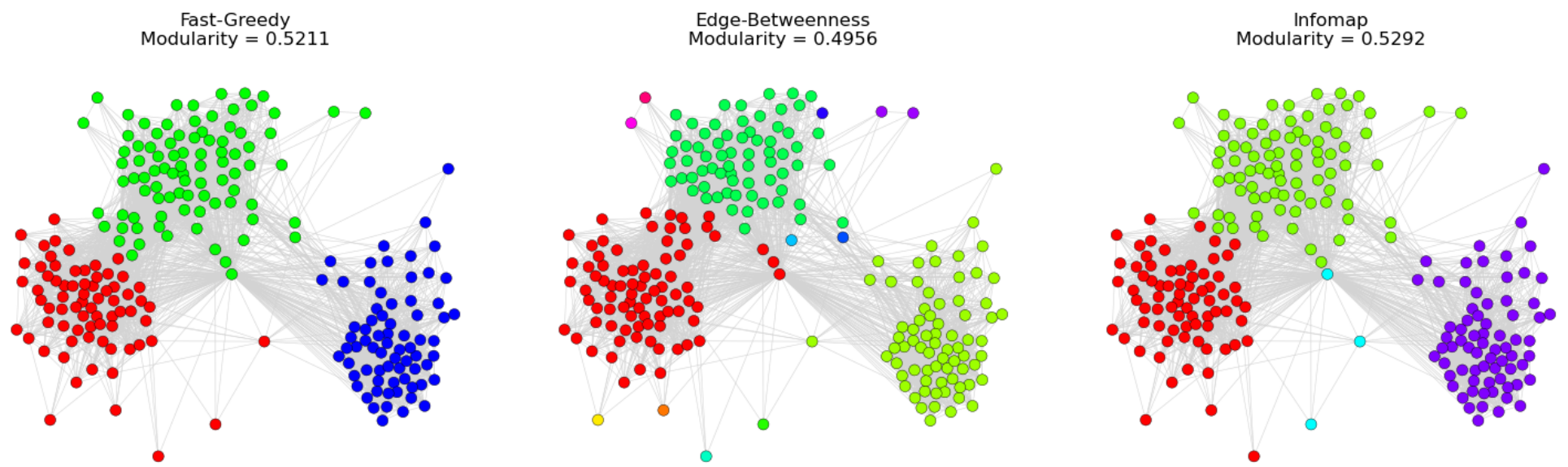
Core Node ID 484 Deleted

Fast-Greedy: 3 Communities, Modularity = 0.5211

Edge-Betweenness: 13 Communities, Modularity = 0.4956

Infomap: 4 Communities, Modularity = 0.5292

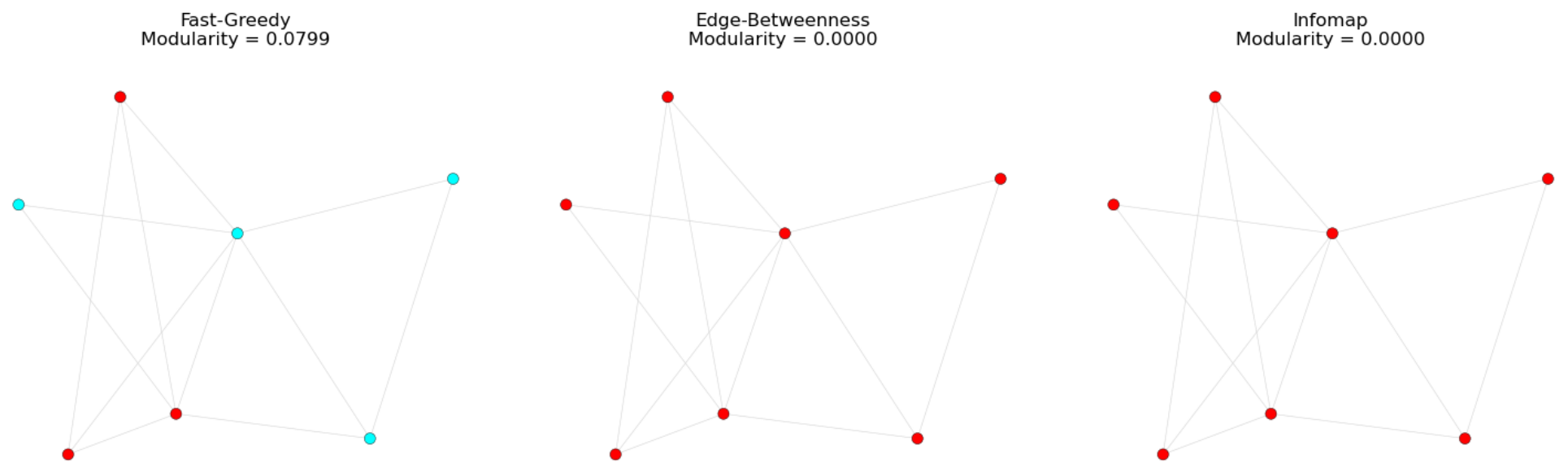
Core Node ID 484 Deleted Modified Personalized Network



In [21]: `plot_node_v2(1087)`

Core Node ID 1088 Deleted
 Fast-Greedy: 2 Communities, Modularity = 0.0799
 Edge-Betweenness: 1 Communities, Modularity = 0.0000
 Infomap: 1 Communities, Modularity = 0.0000

Core Node ID 1088 Deleted Modified Personalized Network



Comparing the Modularity from the Communities in Question 9 to those in Question 10, there wasn't a big difference in the change in modularity. In other words, the modularity score remained almost the same after removing the core node. This suggests there may be high internal redundancy with communities already exhibiting high connectedness, even with the removal of a core node.

3.3 Characteristic of nodes in the personalized network

Question 11

Embeddedness of a node is defined as the number of mutual friends a node shares with the core node. Since a core node's personalized network is made up of the core node and all of the core node's neighbors, all of the neighbors of a non-core node are also neighbors of a core node (thus those are the mutual friends). In other words, the degree of non-core nodes can tell us about the embeddedness between a core node and non-core node.

More specifically, for a non-core node v in core node's personalized network PN , we can define embeddedness between the core node and non-core node as:

$$Embeddedness(v) = deg(v) - 1$$

We subtract one since we exclude the edge connecting v and the core node (only interested in mutual friends).

Question 12

```
In [22]: def embeddedness(pn, v, core=0):
  return pn.degree(v) - 1

def dispersion(pn, v, core=0):
  C = list(set(pn.neighbors(core)) & set(pn.neighbors(v)))
  if len(C) < 2: #If only one mutual or no mutuals then
    return 0
  C_removed = [x for x in range(pn.vcount()) if x not in (core, v)] #Remove v and core node
  C_removed_pn = pn.induced_subgraph(C_removed) #Create modified network without v and core node

  #Remap the indices
  remap_idx = {old: new for new, old in enumerate(C_removed)}
  mutual_idx = [remap_idx[m] for m in C]

  #Get all mutual friends distances
  dist_mat = np.array(C_removed_pn.distances(source=mutual_idx, target=mutual_idx), dtype = float)
  dist_mat[np.isinf(dist_mat)] = 0 # Remove any nodes with no path between them (otherwise we have inf values)
  return dist_mat.sum() / 2 #dist_mat is symmetric so can sum the matrix and divide by 2 (to avoid double count)

def q12_histogram(core):
  pn = g.induced_subgraph([core] + g.neighbors(core))
```

```

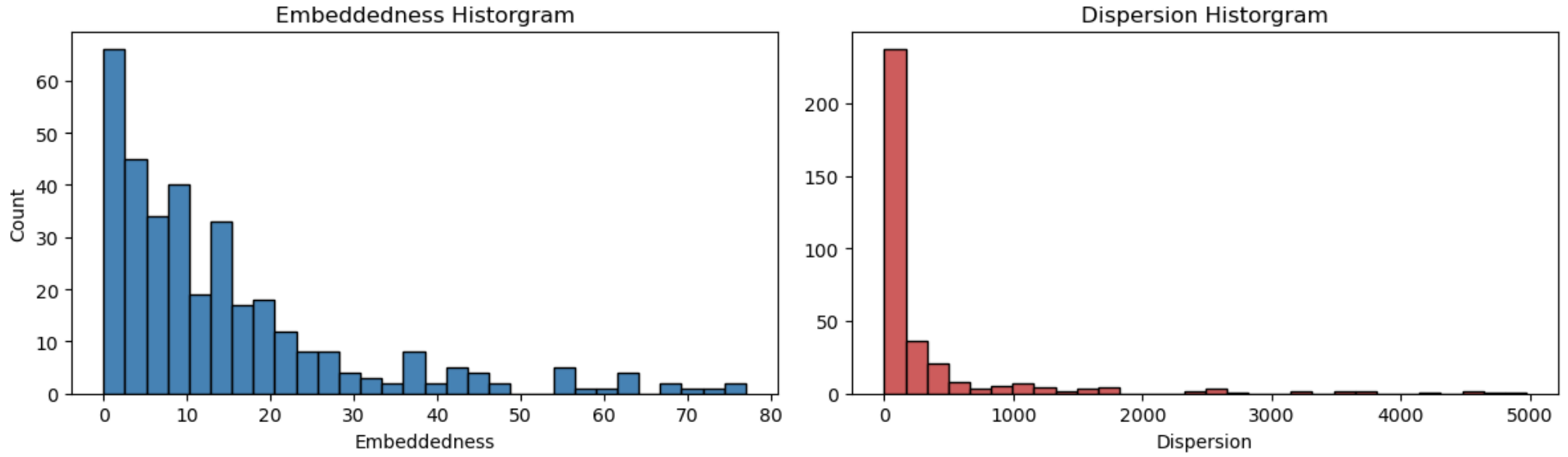
others = [v for v in range(pn.vcount()) if v != 0]
emb = [embeddedness(pn, v) for v in others]
disp = [dispersion(pn, v) for v in others]

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].hist(emb, bins=30, color="steelblue", edgecolor="black")
axes[0].set_title("Embeddedness Histogram"); axes[0].set_xlabel("Embeddedness")
axes[0].set_ylabel("Count")
axes[1].hist(disp, bins=30, color="indianred", edgecolor="black")
axes[1].set_title("Dispersion Histogram"); axes[1].set_xlabel("Dispersion")
fig.suptitle(f"Core Node ID {core + 1}", fontsize=14)
plt.tight_layout(); plt.show()

```

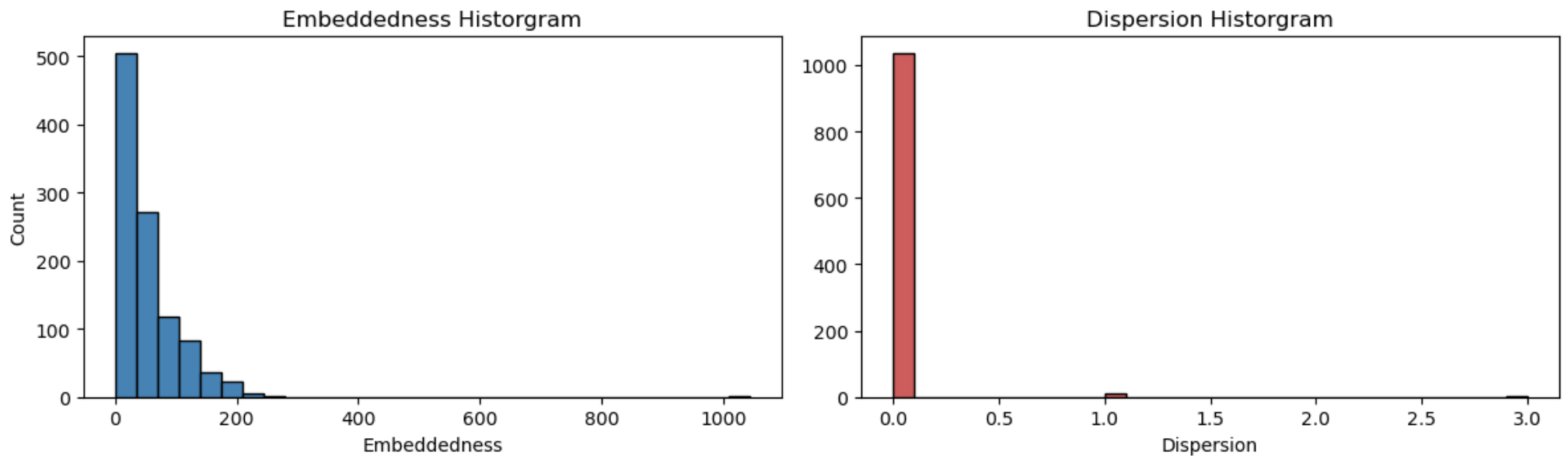
In [23]: q12_histogram(0)

Core Node ID 1



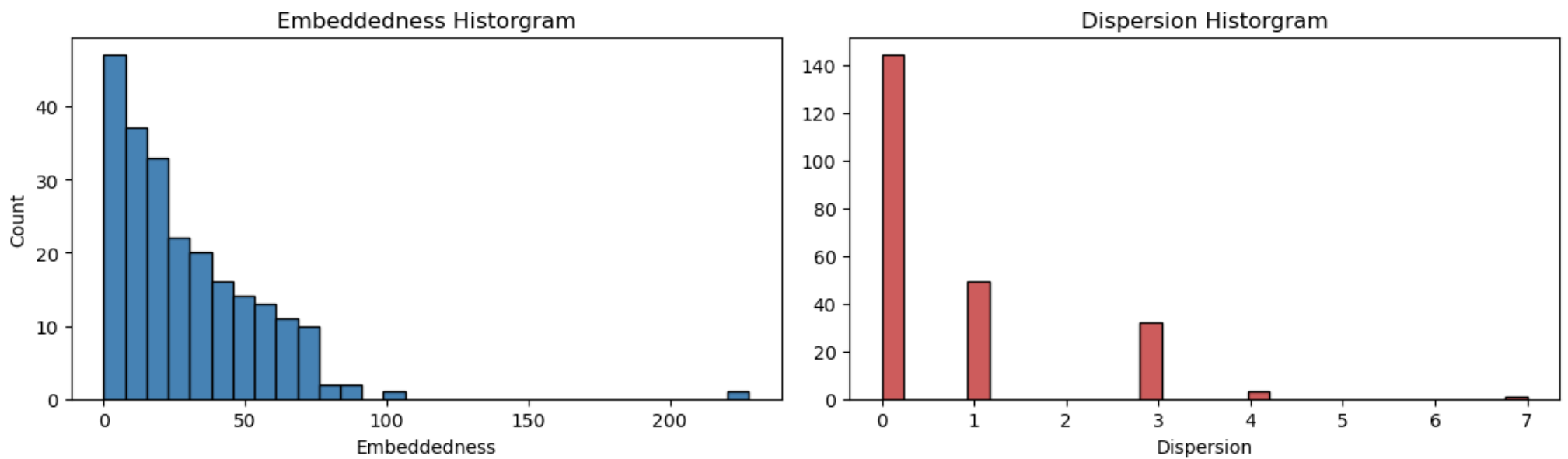
In [24]: q12_histogram(107)

Core Node ID 108



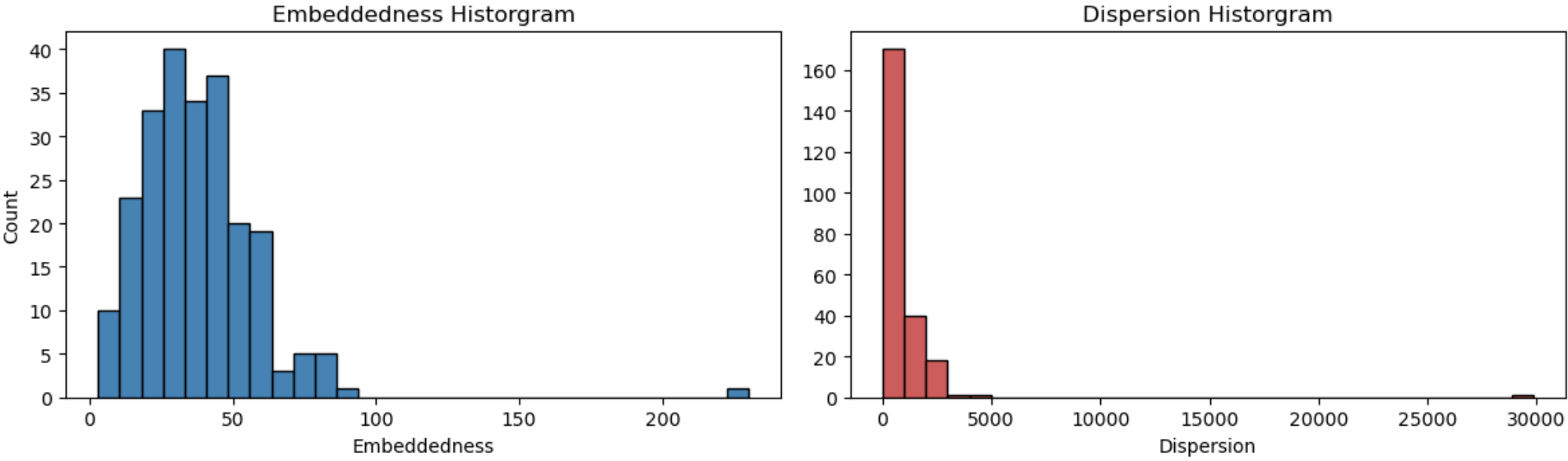
In [25]: q12_histogram(348)

Core Node ID 349



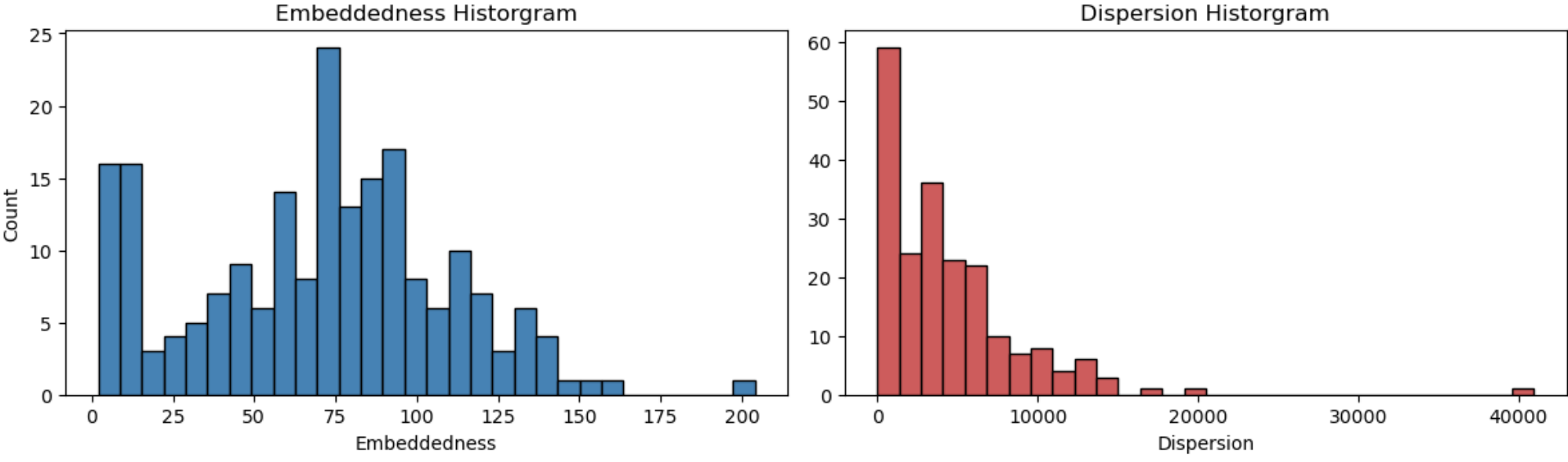
In [26]: q12_histogram(483)

Core Node ID 484



```
In [27]: q12_histogram(1086)
```

Core Node ID 1087



```
In [28]: def highlight_draw_network(pn, clustering, layout, ax, title, highlights):
    pal = ig.RainbowPalette(n=max(len(clustering), 1))

    # Adds color to vertex based on the community they belong
    vcolors = [pal.get(m) for m in clustering.membership]
    vsizes = [10] * pn.vcount()
    vshapes = ["circle"] * pn.vcount()
    for v, col in highlights.items(): #Forces the vertex based on highlights to be highlighted a specific color
        vcolors[v] = col
        vsizes[v] = 32
        vshapes[v] = "diamond"

    # make ALL edges gray(make then shorter width) but force edges incident to highlighted node to be highlighted too(have longer width too)
    ecolors, ewidths = [], []
    for e in pn.es:
        col = highlights.get(e.source) or highlights.get(e.target)
        ecolors.append(col if col else "lightgray")
        ewidths.append(1.6 if col else 0.3)

    ig.plot(
        pn,
        target=ax,
        layout=layout,
        vertex_color=vcolors,
        vertex_size=vsizes,
        vertex_shape=vshapes,
        vertex_frame_width=0.3,
        edge_color=ecolors,
        edge_width=ewidths
    )
    ax.set_title(title)
    ax.set_axis_off()

def q13_plot(core):
    nodes = [core] + g.neighbors(core)
    pn = g.induced_subgraph(nodes)
    layout = pn.layout("fr")
    fg = pn.community_fastgreedy().as_clustering()

    others = [v for v in range(pn.vcount()) if v != 0] #Get all non-core nodes
    disp = {v: dispersion(pn, v) for v in others}
    max_disp = max(disp, key=disp.get)
    print(f"Core Node {core + 1}: Max Dispersion Node = {nodes[max_disp] + 1} (Dispersion = {disp[max_disp]})")

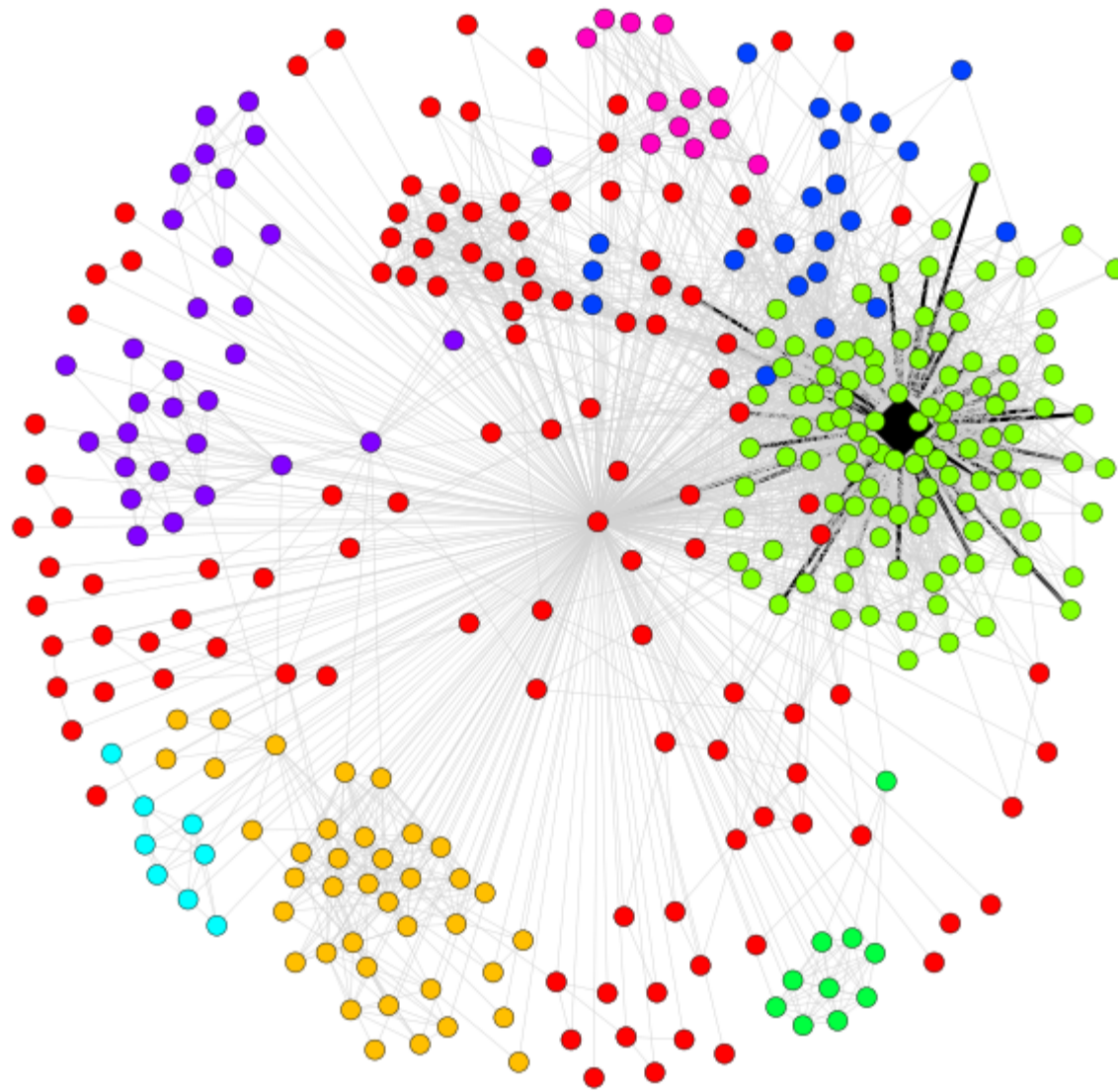
    fig, ax = plt.subplots(figsize=(7, 7))
    highlight_draw_network(pn, fg, layout, ax, f"Core Node {core + 1} - Max Dispersion Highlighted", {max_disp: "black"})
    plt.tight_layout(); plt.show()
```

In the following plots, the node with maximum dispersion is highlighted in black.

```
In [29]: q13_plot(0)
```

```
Core Node 1: Max Dispersion Node = 57 (Dispersion = 4971.0)
```

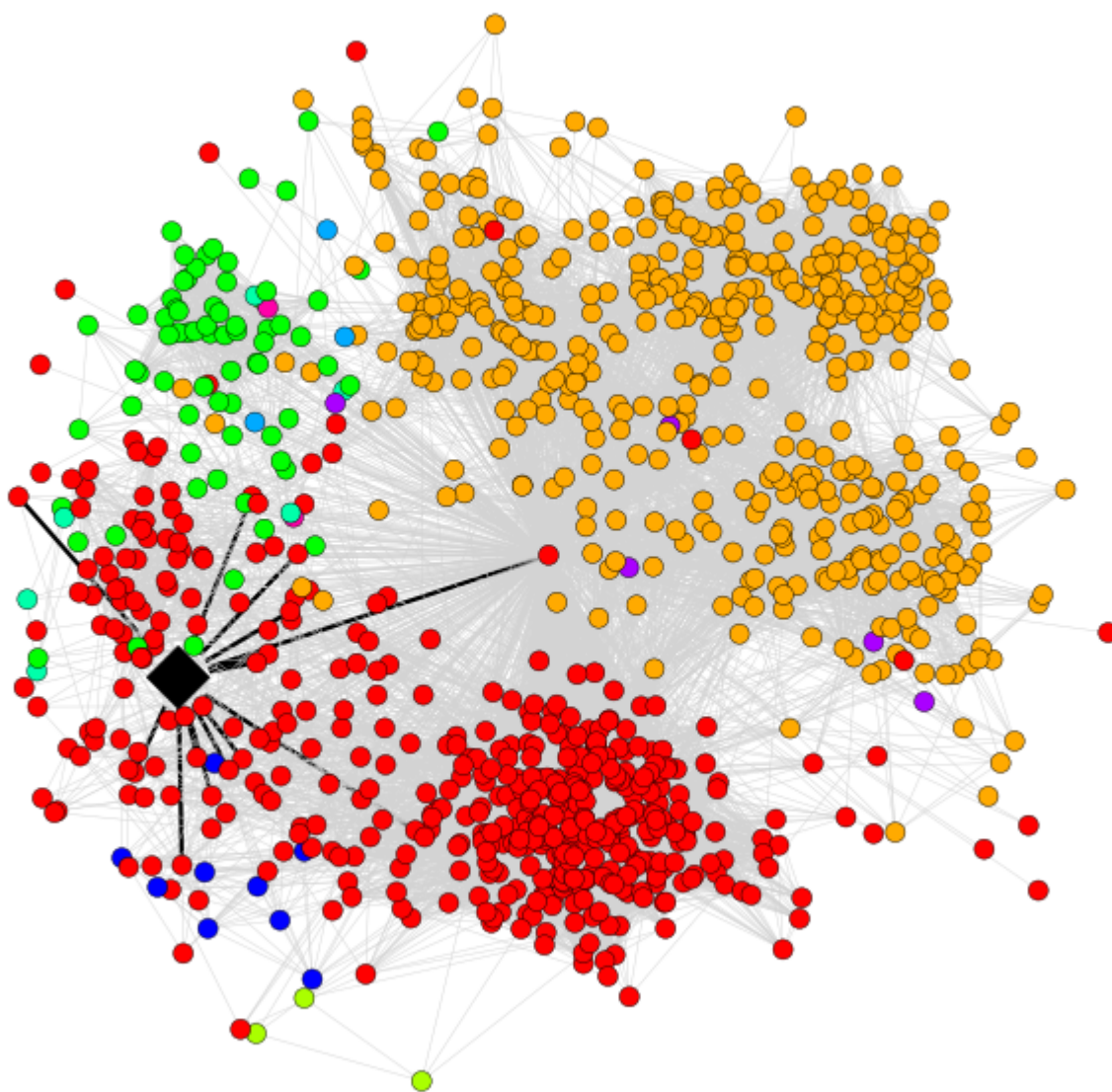
Core Node 1 — Max Dispersion Highlighted



In [30]: `q13_plot(107)`

Core Node 108: Max Dispersion Node = 1685 (Dispersion = 3.0)

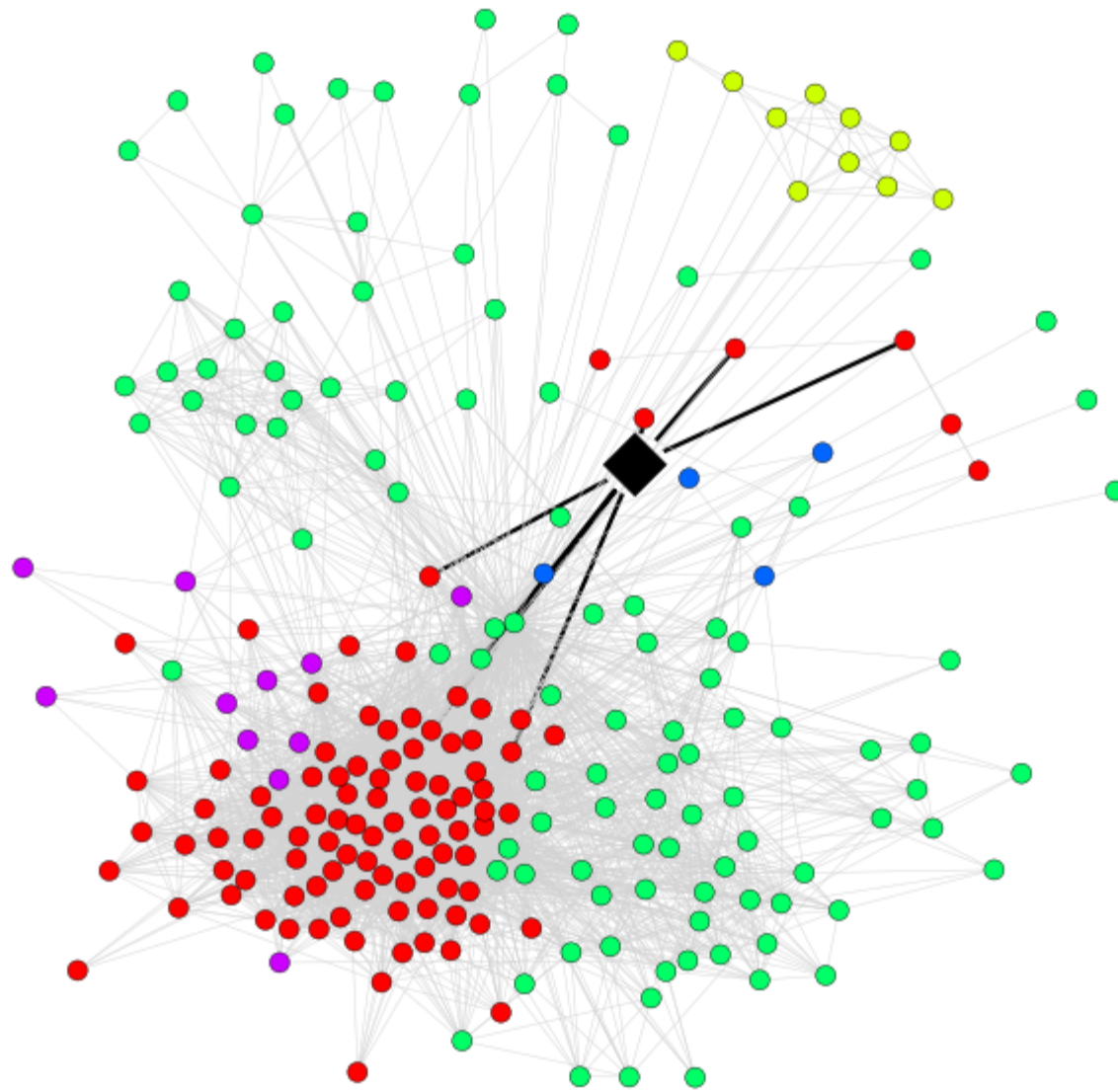
Core Node 108 — Max Dispersion Highlighted



In [31]: `q13_plot(348)`

Core Node 349: Max Dispersion Node = 428 (Dispersion = 7.0)

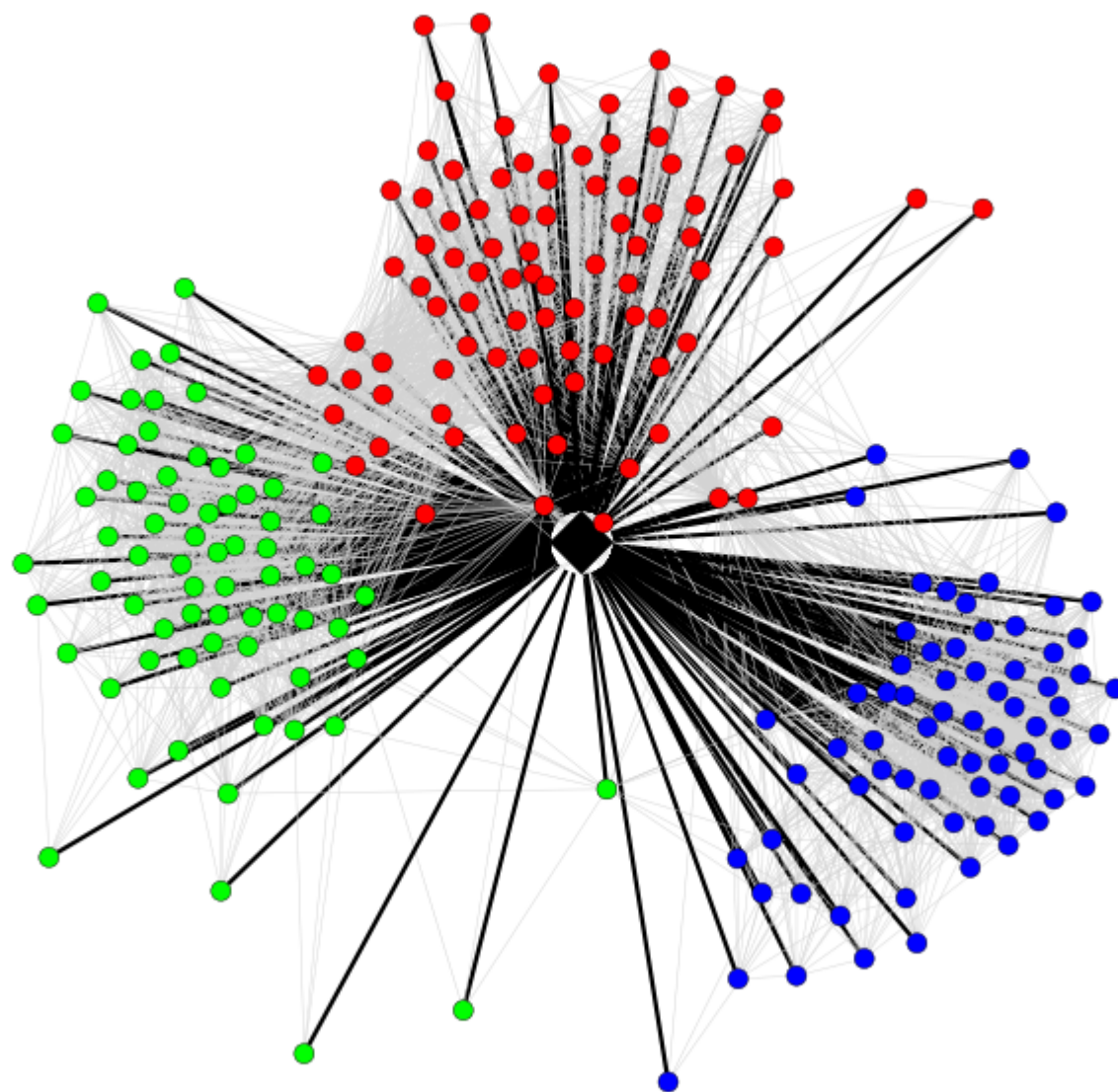
Core Node 349 — Max Dispersion Highlighted



In [32]: `q13_plot(483)`

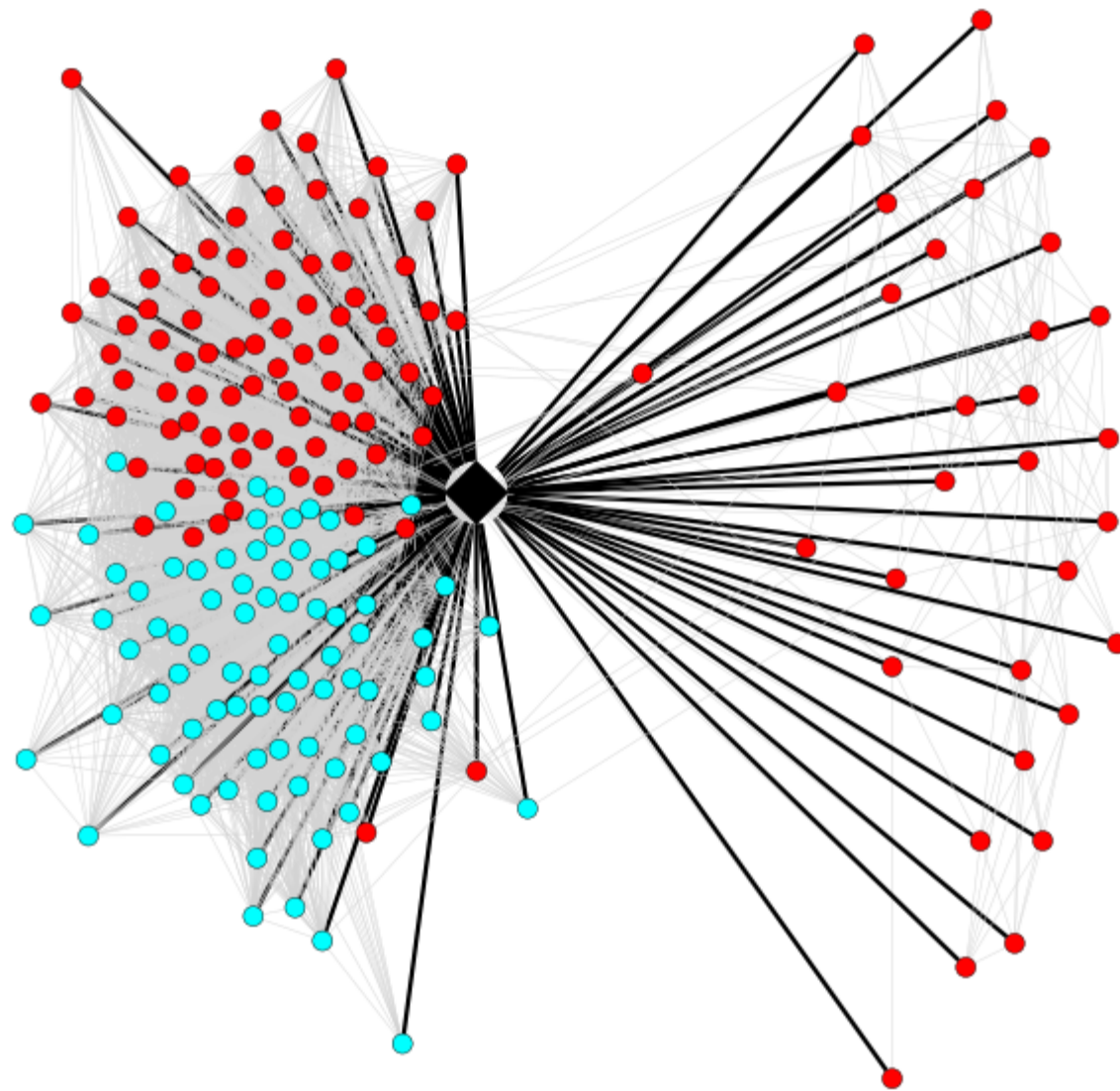
Core Node 484: Max Dispersion Node = 483 (Dispersion = 29837.0)

Core Node 484 — Max Dispersion Highlighted



In [33]: `q13_plot(1086)`

Core Node 1087: Max Dispersion Node = 1084 (Dispersion = 40969.0)



Question 14

```
In [34]: def q14_plot(core):
    nodes = [core] + g.neighbors(core)
    pn = g.induced_subgraph(nodes)
    layout = pn.layout("fr")
    fg = pn.community_fastgreedy().as_clustering()

    others = [v for v in range(pn.vcount()) if v != 0] #Get all non-core nodes
    emb = {v: embeddedness(pn, v) for v in others}
    disp = {v: dispersion(pn, v) for v in others}

    zero_emb = [v for v in others if emb[v] > 0] # exclude zero embeddedness
    max_emb = max(zero_emb, key=lambda v: emb[v])
    max_ratio = max(zero_emb, key=lambda v: disp[v] / emb[v])

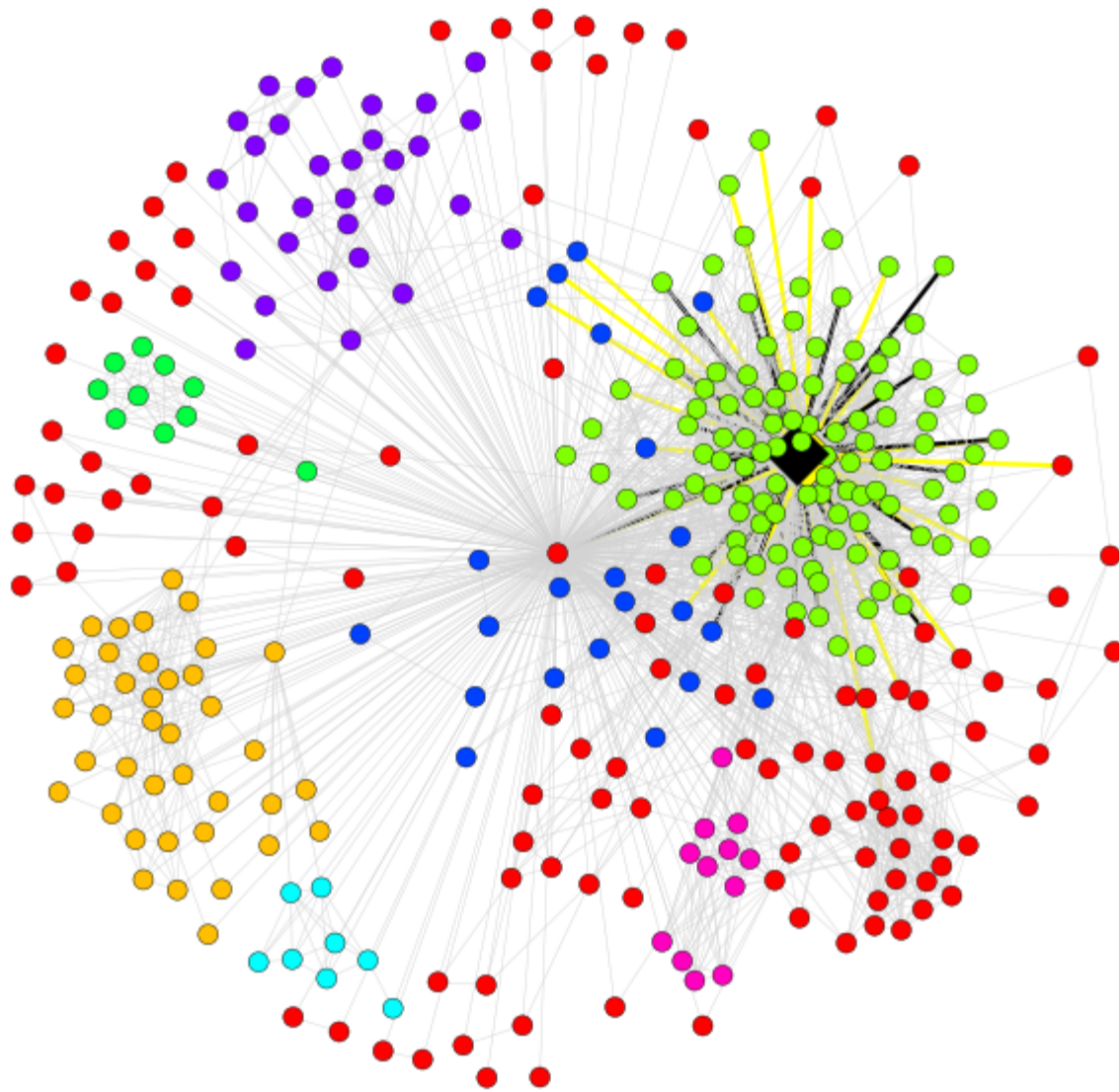
    print(f"Core Node {core + 1}:")
    print(f"Max Embeddedness Node = {nodes[max_emb] + 1} "f"(Embeddedness = {emb[max_emb]})")
    print(f"Max Dispersion/Embeddedness Node = {nodes[max_ratio] + 1} "f"(ratio = {disp[max_ratio]/emb[max_ratio]})")
    if nodes[max_emb] == nodes[max_ratio]:
        print("Max Embeddedness Node and Max Dispersion/Embeddedness Node are the same for this Core Node's Personalized Network")

    fig, ax = plt.subplots(figsize=(7, 7))
    highlight_draw_network(pn, fg, layout, ax, f"Core Node {core + 1} - Max Embeddedness (black) "f"& Max Dispersion/Embeddednes (yellow)",
    plt.tight_layout(); plt.show()
```

```
In [35]: q14_plot(0)
```

```
Core Node 1:
Max Embeddedness Node = 57 (Embeddedness = 77)
Max Dispersion/Embeddedness Node = 26 (ratio = 66.22058823529412)
```


Core Node 1 — Max Embeddedness (black) & Max Dispersion/Embeddednes (yellow)



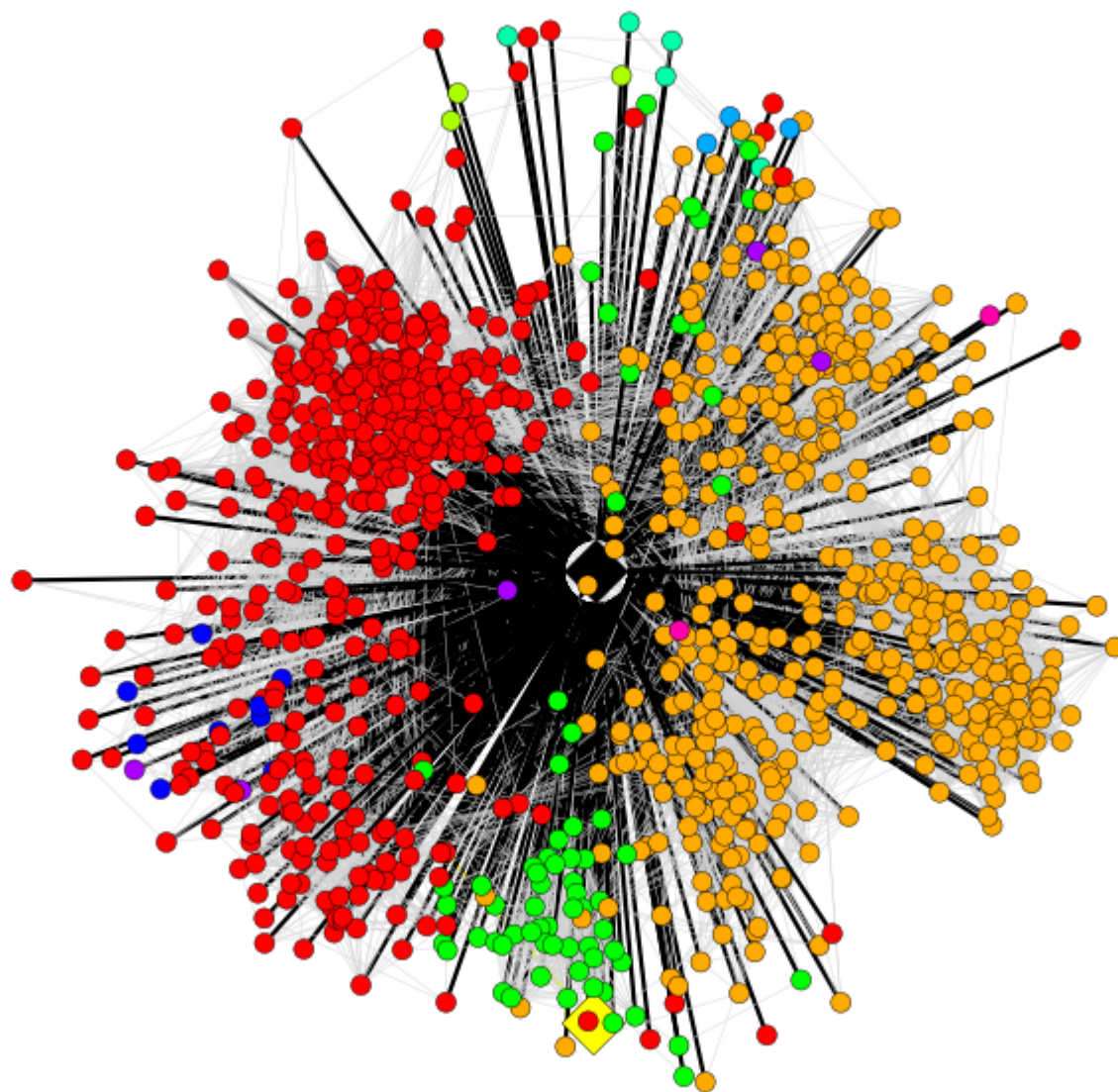
In [36]: `q14_plot(107)`

Core Node 108:

Max Embeddedness Node = 59 (Embeddedness = 1044)

Max Dispersion/Embeddedness Node = 905 (ratio = 0.5)

Core Node 108 — Max Embeddedness (black) & Max Dispersion/Embeddednes (yellow)



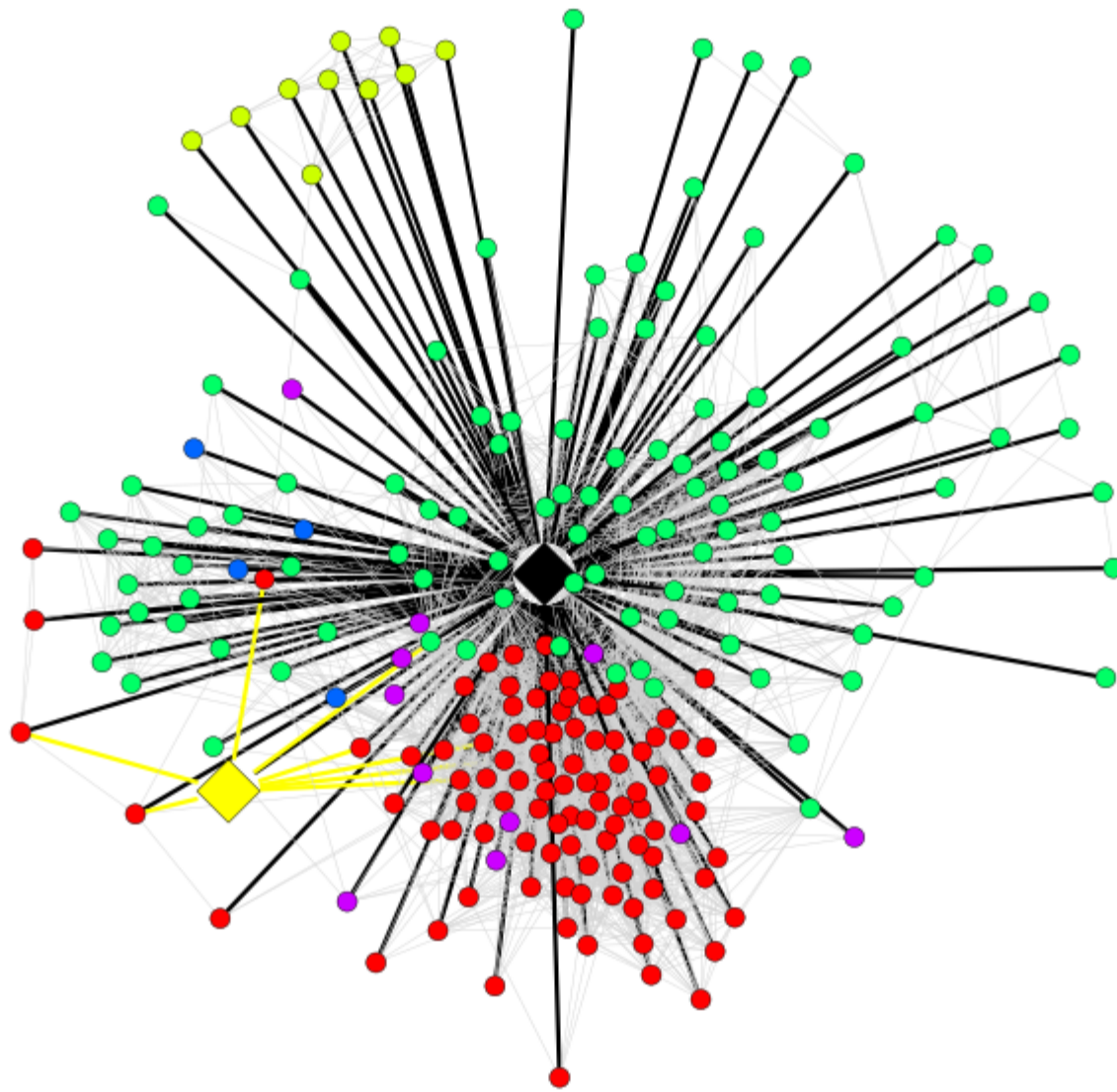
In [37]: `q14_plot(348)`

Core Node 349:

Max Embeddedness Node = 199 (Embeddedness = 228)

Max Dispersion/Embeddedness Node = 428 (ratio = 0.875)

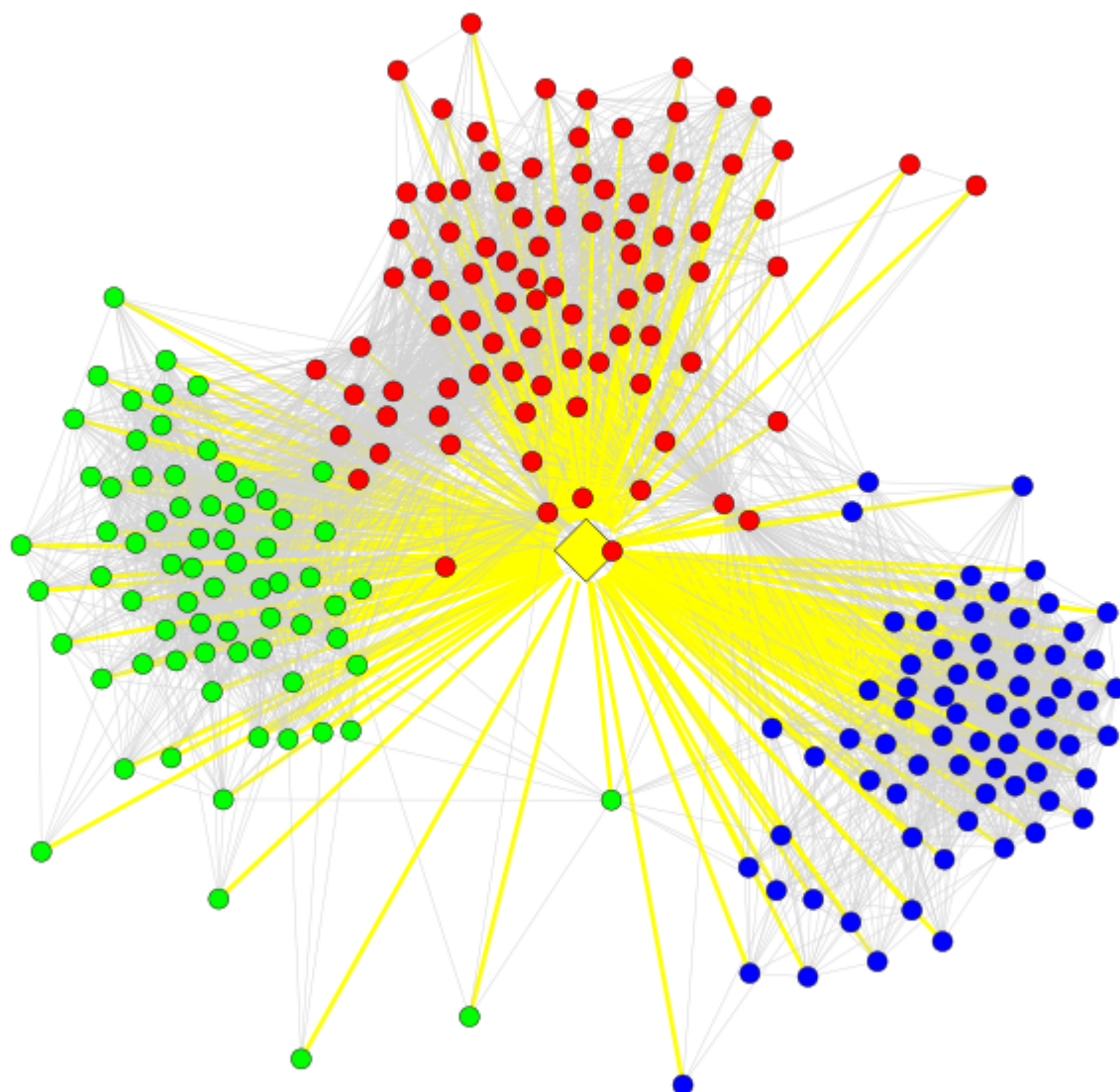
Core Node 349 — Max Embeddedness (black) & Max Dispersion/Embeddednes (yellow)



In [38]: `q14_plot(483)`

Core Node 484:
Max Embeddedness Node = 483 (Embeddedness = 230)
Max Dispersion/Embeddedness Node = 483 (ratio = 129.72608695652173)
Max Embeddedness Node and Max Dispersion/Embeddedness Node are the same for this Core Node's Personalized Network

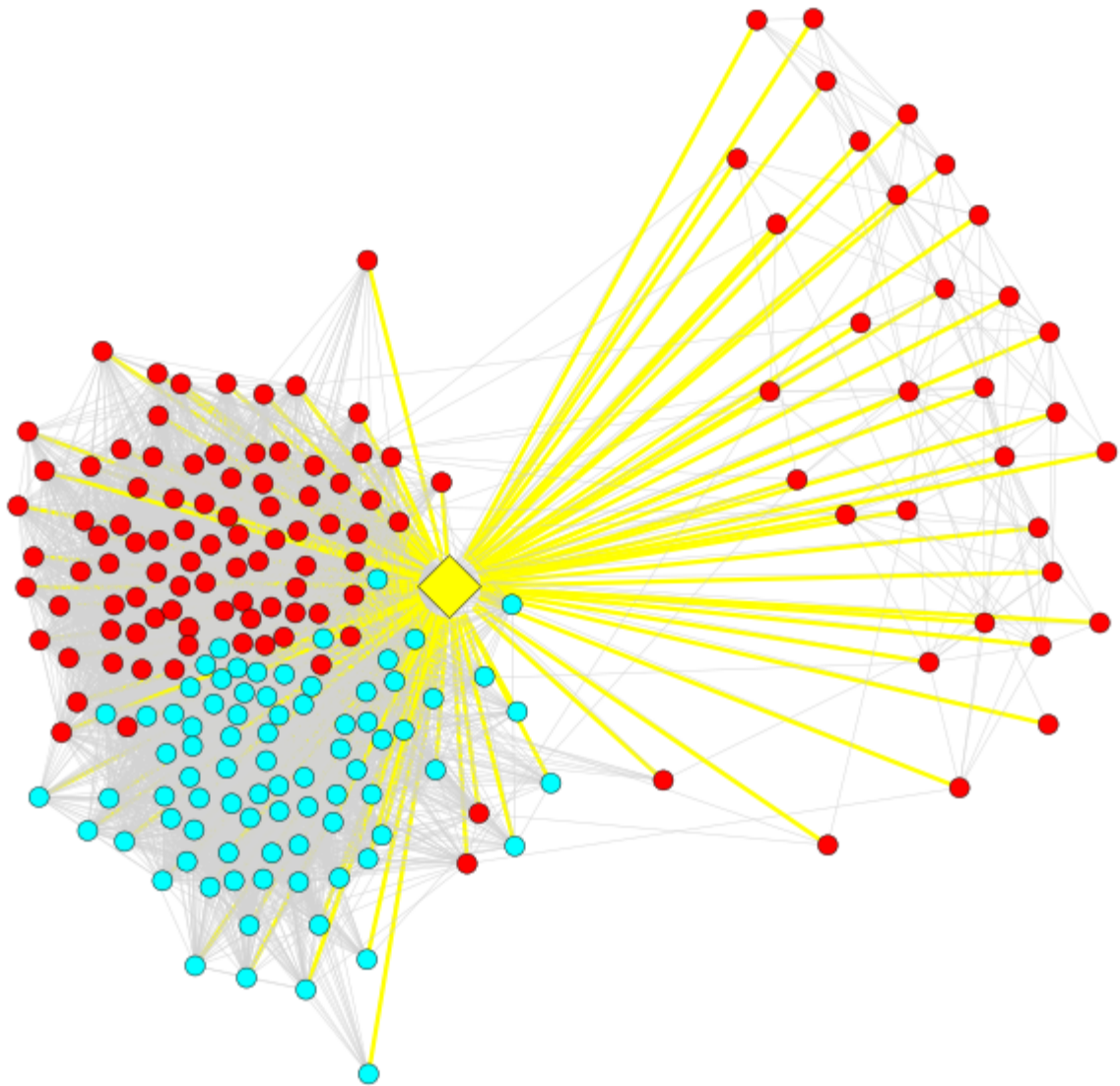
Core Node 484 — Max Embeddedness (black) & Max Dispersion/Embeddednes (yellow)



In [39]: `q14_plot(1086)`

Core Node 1087:
Max Embeddedness Node = 1084 (Embeddedness = 204)
Max Dispersion/Embeddedness Node = 1084 (ratio = 200.82843137254903)
Max Embeddedness Node and Max Dispersion/Embeddedness Node are the same for this Core Node's Personalized Network

Core Node 1087 — Max Embeddedness (black) & Max Dispersion/Embeddednes (yellow)



Question 15

For the Max Embeddedness node, either of two scenarios happen: they are inside one dense community and its incident edges stay almost entirely within that one community (like Core Node 1) or the node is right in the middle of the network with edges to almost all nodes around it (like the rest of the core nodes_. So embeddedness reveals how central a node is inside one cohesive group. The biggest cluster tends to contain the most embedded node because there are more "mutual friends" there.

For dispersion, a high-dispersion node is one where the common neighbors of that node and the core node are not well connected to each other. After you remove those two nodes from the graph, those common neighbors end up far apart. A low-dispersion node is the opposite.

Dispersion/Embeddedness combines both: It is high when the "mutual friends" are spread far apart (high dispersion) relative to how many "mutual friends" there are (embeddedness).

4. Friend Recommendation in Personalized Networks

```
In [40]: pn_415 = g.induced_subgraph([414] + g.neighbors(414))
```

Question 16

```
In [41]: nodes_degree_24 = pn_415.vs.select(_degree=24)
Nr = [v.index for v in nodes_degree_24]
print("|N_r| = ",len(Nr))
print(Nr)
```

|N_r| = 11
[30, 52, 74, 89, 92, 101, 117, 132, 133, 135, 136]

Question 17

```
In [42]: def neighborhood_measure(pn, measure):
    if measure == "common":
        A = np.array(pn.get_adjacency().data)
        return A @ A
    if measure == "jaccard":
        return np.array(pn.similarity_jaccard())
    if measure == "adamic":
        return np.array(pn.similarity_inverse_log_weighted())
    raise ValueError(measure)

def avg_accuracy(pn, Nr, measure, trials=10, p=0.25, seed=0):
    rng = random.Random(seed)

    all_Nr_accuracies = []
```

```

for user_i in Nr:
    accuracies = []
    for trial in range(trials):
        pn_copy = pn.copy() # Make a copy since we will remove edges from node i at random

        # 1. Remove each edge of node i with prob 0.25
        remove_edges = [edge for edge in pn_copy.incident(user_i) if rng.random() < p]
        if not remove_edges:
            continue
        R_i = set() # Create the list of friends deleted Ri (if there are no edges removed, Ri is empty)
        for edge in remove_edges:
            edge_sequence = pn_copy.es[edge]
            if edge_sequence.source == user_i: #Makes sure user i is not deleted, but the node connecting to i
                R_i.add(edge_sequence.target)
            else:
                R_i.add(edge_sequence.source)
        pn_copy.delete_edges(remove_edges)

        # 2. Recommend |R_i| new friends to user i
        score = neighborhood_measure(pn_copy, measure)[user_i] #Neighborhood_measure returns a matrix, [user_i] gets you the ith row

        neighbors_after_removal = set(pn_copy.neighbors(user_i)) #Get the neighbors after removing edges
        recommend = [j for j in range(pn_copy.vcount()) if j != user_i and j not in neighbors_after_removal] # Get the possible candidates
        recommend.sort(key=lambda j: score[j], reverse=True) #Sort friends based on highest measure/score
        P_i = set(recommend[:len(R_i)]) # Get P_i; List of friends recommended

        # 3. Compute accuracies
        accuracies.append(len(P_i & R_i) / len(R_i))
    if accuracies: # Just in case R_i is empty
        all_Nr_accuracies.append(sum(accuracies) / len(accuracies))
return sum(all_Nr_accuracies) / len(all_Nr_accuracies)

```

```
In [43]: print("Common Neighbors Average Accuracy = ", avg_accuracy(pn_415, Nr, "common"))
```

Common Neighbors Average Accuracy = 0.8157523284796011

```
In [44]: print("Jaccard Average Accuracy = ", avg_accuracy(pn_415, Nr, "jaccard"))
```

Jaccard Average Accuracy = 0.798429970702698

```
In [45]: print("Adamic Adar Average Accuracy = ", avg_accuracy(pn_415, Nr, "adamic"))
```

Adamic Adar Average Accuracy = 0.8216614193886922

Adamic Adar Measure had the highest accuracy; thus, the best friend recommendation algorithm

2. Google+ Network

```
In [60]: import os
import os.path as path
import glob
import igraph as ig

def gplus_personal_networks(verbose=False):
    """
    Reads the directory gplus/ files and creates personal networks.
    """
    # Path to gplus/ directory
    current_dir = path.join(os.getcwd(), 'gplus')

    # Find all .circles files
    circles_files = glob.glob(os.path.join(current_dir, "*.circles"))

    if verbose: print(f"Found {len(circles_files)} total ego nodes")

    # Iterate through each user
    networks = {}
    for circles_fpath in circles_files:
        # Extract nodeId from filename (e.g., "12345.circles" -> "12345")
        filename = path.basename(circles_fpath)
        node_id = filename.split(".")[0] # Ego node

        # Count number of circles for this ego node
        with open(circles_fpath, "r") as f:
            circles = f.readlines()

        # Only consider those with 2+ circles
        if len(circles) <= 2:
            if verbose: print(f"Node id {node_id} had <= 2 circles")
            continue

        if verbose: print(f"Ego Node: {node_id} ({len(circles)} circles found)")

        # Parse same node's .edges file
        edges_fpath = path.join(current_dir, f"{node_id}.edges")
        if not path.exists(edges_fpath) or path.getsize(edges_fpath) == 0:
            # Skip if the file is empty or missing
            continue

        # Read network connections
        network_edges = []
        unique_nodes = set()
        with open(edges_fpath, "r") as f:
            for line in f:
                # gplus edges are directed, 'a follows b' (a -> b)
                source, target = line.strip().split()
                network_edges.append((source, target))
                unique_nodes.add(source)
                unique_nodes.add(target)

        # Add ego node and its connections
        # .readme says ego node doesn't appear in .edges file, but
        # make the ego node follow every node id that appears in file
        unique_nodes.add(node_id)
        for neighbor in unique_nodes:
            if neighbor != node_id:
                network_edges.append((node_id, neighbor)) # Ego follows neighbor

        # Map string IDs to unique integer IDs for igraph compatibility
        # (igraph requires internal vertex indices to be sequential integers)
        sorted_nodes = sorted(list(unique_nodes))
        id_map = {node_id: idx for idx, node_id in enumerate(sorted_nodes)}
        mapped_edges = [(id_map[src], id_map[tgt]) for src, tgt in network_edges]

        # Create directed graph
        g = ig.Graph(directed=True)
        g.add_vertices(len(unique_nodes))
        g.add_edges(mapped_edges)
        if verbose: print(f"    Created personal network with {g.vcount()} nodes and {g.ecount()} directed edges.")

        # Add extra data for later
        # Store original SNAP IDs
        # Organized like Pandas DataFrame. Index is a node and has columns/attributes (e.g., snap_id)
        g.vs["snap_id"] = sorted_nodes # Assigns each vertex to one node
        # Flag ego node explicitly
        g.vs["is_ego"] = [uid == node_id for uid in sorted_nodes]

        # Store for output
        networks[node_id] = g

    return networks
```

Question 18

```
In [13]: gplus_networks = gplus_personal_networks()
```



```
n_networks = len(gplus_networks)
print(f"There are {n_networks} personal networks")
```

There are 57 personal networks

Question 19

```
In [27]: import matplotlib.pyplot as plt

def plot_degree_distributions(node_id, networks):
    """
    Plots the in-degree and out-degree distributions of a personal network.
    """

    # In-degrees and out-degrees for all vertices in specific network
    g = networks[node_id]
    in_degrees = g.indegree()
    out_degrees = g.outdegree()

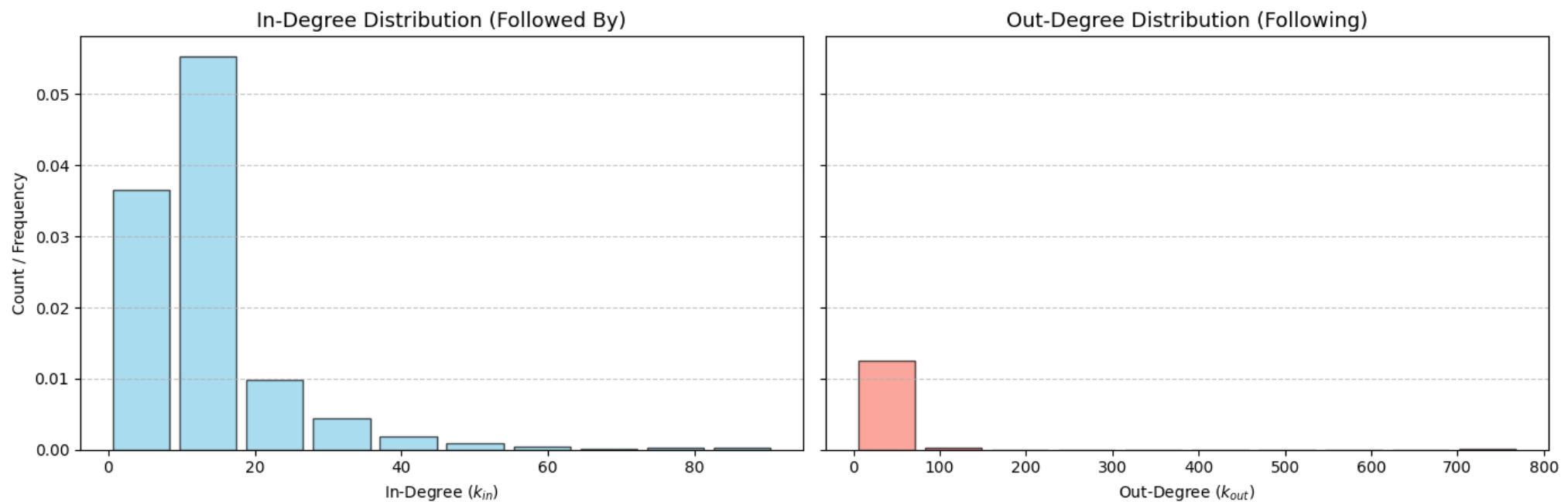
    # Plot
    fig, axes = plt.subplots(1, 2, figsize=(14, 5), sharey=True)
    fig.suptitle(f"Degree Distributions for Personal Network: {node_id}", fontsize=16, fontweight='bold')

    # In-degree
    axes[0].hist(in_degrees, density=True, color="skyblue", edgecolor="black", alpha=0.7, rwidth=0.85)
    axes[0].grid(axis='y', linestyle='--', alpha=0.7) # Clean background gridlines
    axes[0].set_title("In-Degree Distribution (Followed By)", fontsize=13)
    axes[0].set_xlabel("In-Degree ( $k_{in}$ )")
    axes[0].set_ylabel("Count / Frequency")

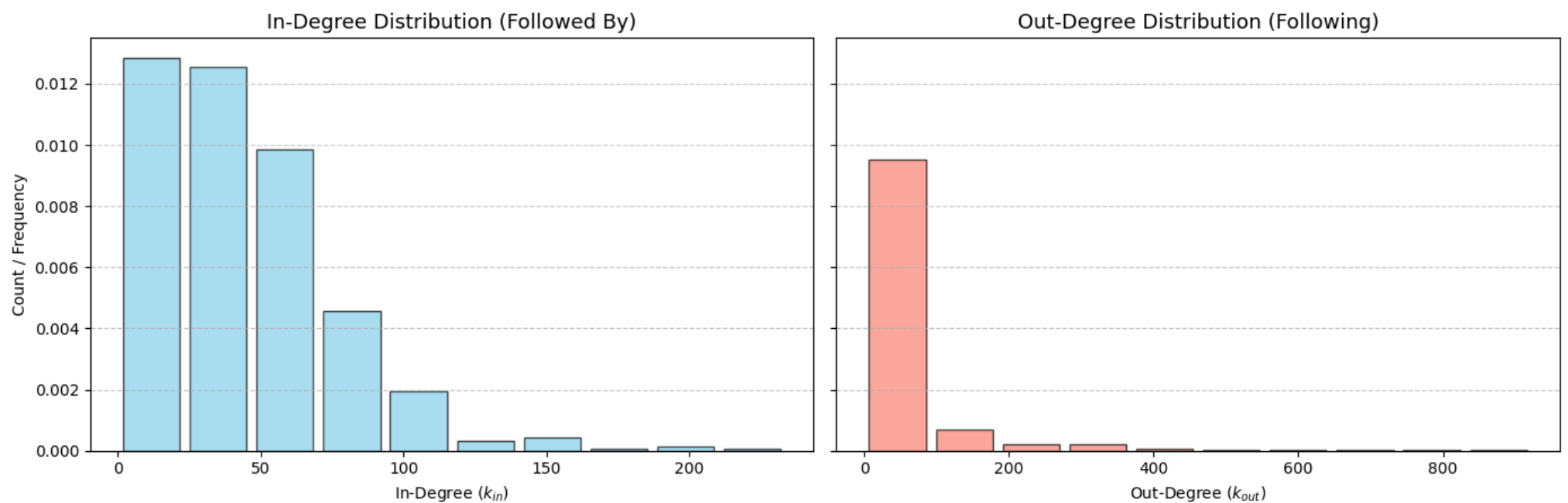
    # Out-degree
    axes[1].hist(out_degrees, density=True, color="salmon", edgecolor="black", alpha=0.7, rwidth=0.85)
    axes[1].grid(axis='y', linestyle='--', alpha=0.7)
    axes[1].set_title("Out-Degree Distribution (Following)", fontsize=13)
    axes[1].set_xlabel("Out-Degree ( $k_{out}$ )")
    plt.tight_layout()
    plt.show()
```

```
In [28]: specific_ids = ['109327480479767108490', '115625564993990145546', '101373961279443806744']
for myid in specific_ids:
    plot_degree_distributions(myid, gplus_networks)
```

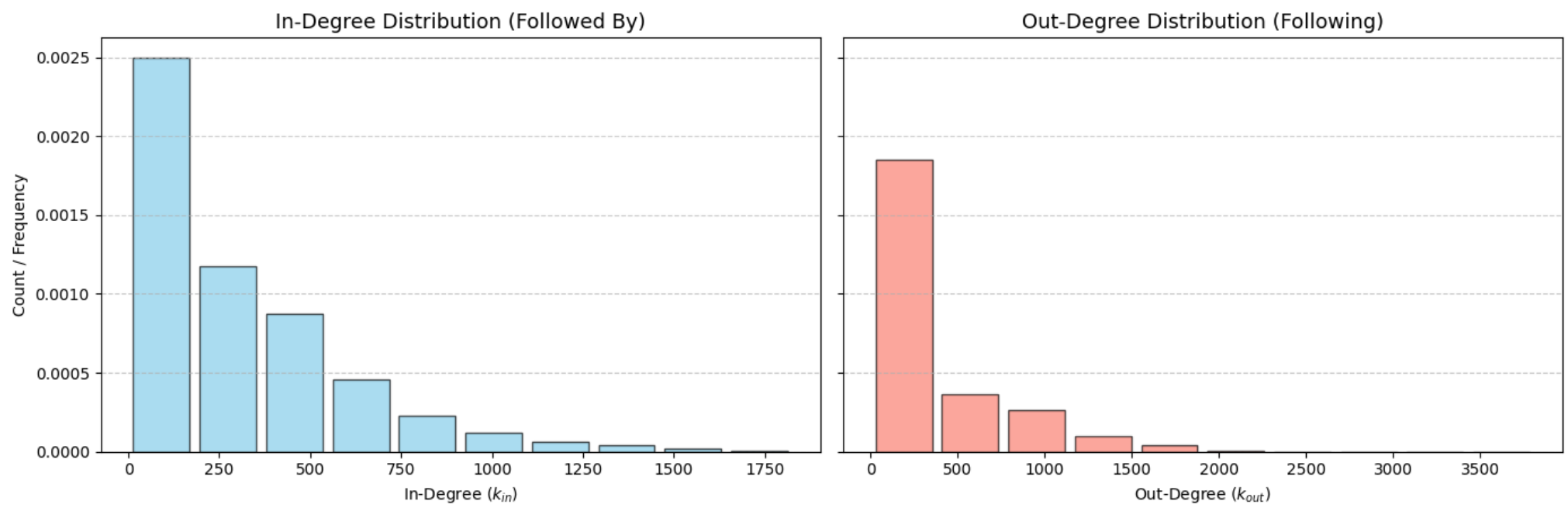
Degree Distributions for Personal Network: 109327480479767108490



Degree Distributions for Personal Network: 115625564993990145546



Degree Distributions for Personal Network: 101373961279443806744



The 3 personal networks above do not have similar in- and out-degree distributions.

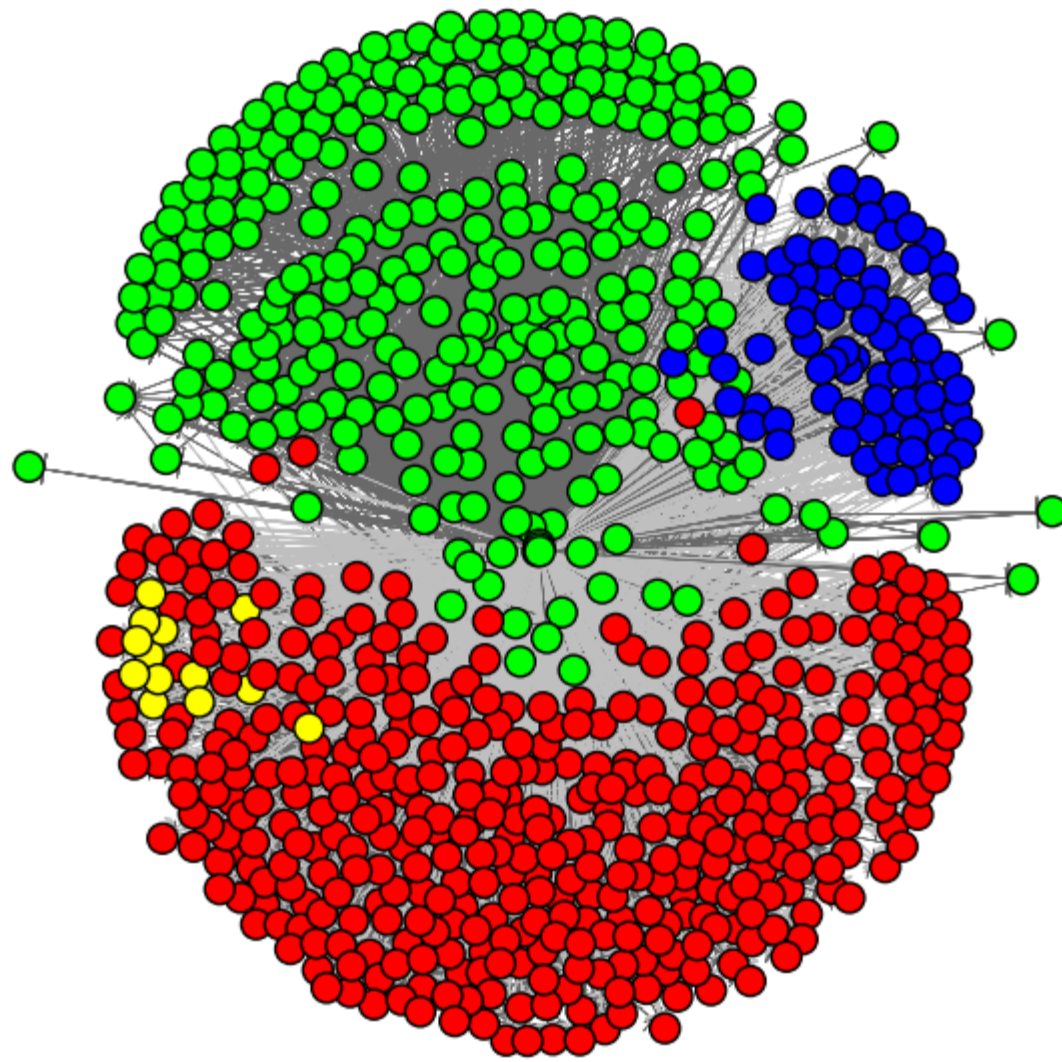
1. Community structure of personal networks

Question 20

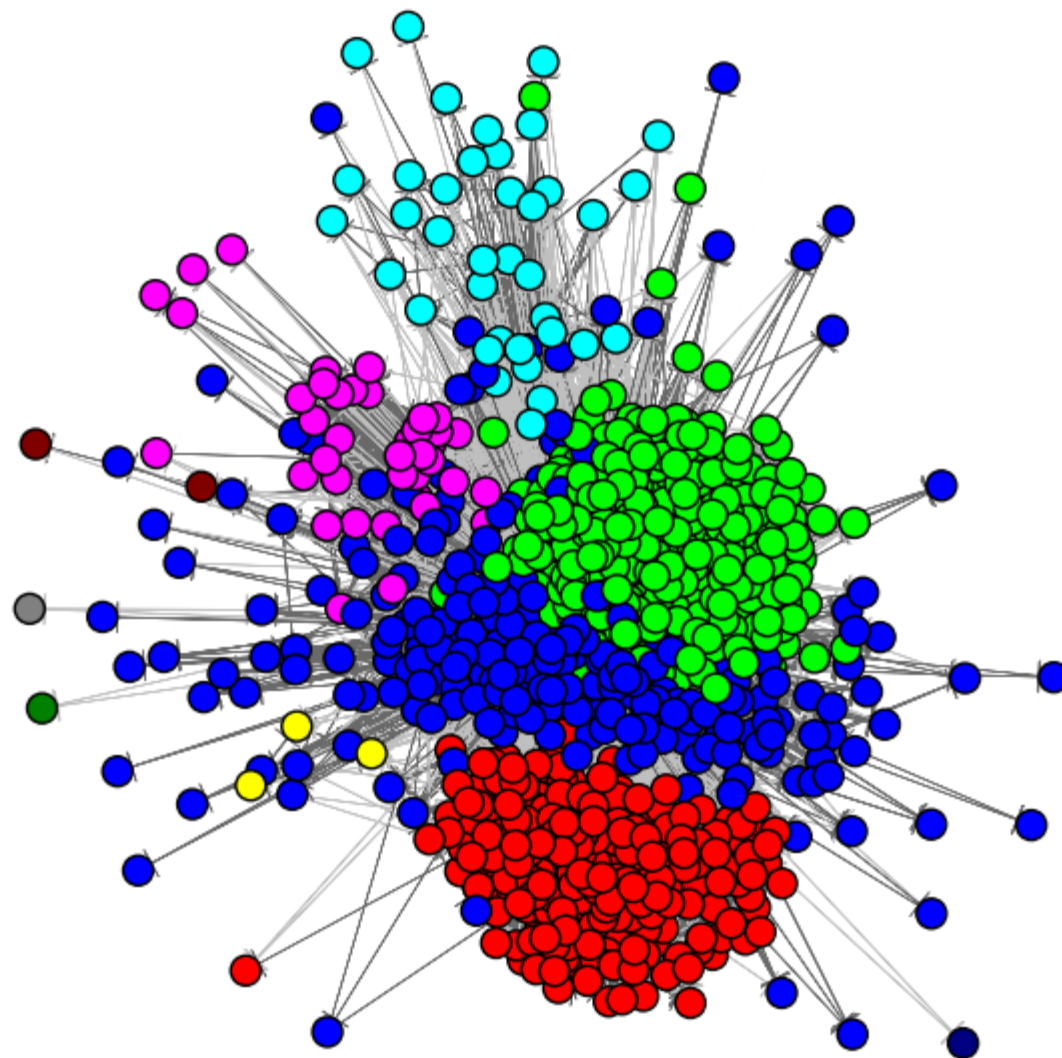
```
In [29]: def plot_walktrap_communities(node_id, gplus_networks):  
    ...  
    Performs a Walktrap community detection algorithm for a given node.  
    ...  
    g = gplus_networks[node_id]  
    # Run Walktrap  
    dendrogram = g.community_walktrap()  
    # Returns partition cut at the maximum index modularity  
    communities = dendrogram.as_clustering()  
    # Modularity scores  
    modularity_score = communities.modularity  
  
    # Plot communities  
    fig, ax = plt.subplots(figsize=(8, 8))  
    ig.plot(  
        communities,  
        target=ax,  
        vertex_size=15,  
        vertex_label=None, # Hide labels so the plot isn't cluttered  
        edge_arrow_size=0.5,  
        edge_width=0.5  
    )  
  
    ax.set_title(f"Walktrap Communities for Ego: {node_id}\nModularity (Q) = {modularity_score:.4f}",  
                fontsize=14, fontweight='bold')  
    plt.show()  
  
    return modularity_score
```

```
In [33]: # Walktrap community detection algorithm  
specific_ids = ['109327480479767108490', '115625564993990145546', '101373961279443806744']  
  
modularity_scores = []  
for node_id in specific_ids:  
    mod_score = plot_walktrap_communities(node_id, gplus_networks)  
    modularity_scores.append(mod_score)  
  
print('Modularity Scores:\n-----')  
for node_id, mscore in zip(specific_ids, modularity_scores):  
    print(f'Node ID {node_id}: Modularity score {mscore:.4f}')
```

Walktrap Communities for Ego: 109327480479767108490
Modularity (Q) = 0.2753



Walktrap Communities for Ego: 115625564993990145546
Modularity (Q) = 0.3200



Walktrap Communities for Ego: 101373961279443806744

Modularity (Q) = 0.1920



Modularity Scores:

Node ID 109327480479767108490: Modularity score 0.2753

Node ID 115625564993990145546: Modularity score 0.3200

Node ID 101373961279443806744: Modularity score 0.1920

The last plot looks smushed because there is poor community separation as indicated by the low modularity index value in comparison to the previous two. In other words, there is likely just a single large giant component, which explains why the plot is smushed into a single foci.

Question 21

Homogeneity is a measure of how similar the communities partitioned by an algorithm are to the actual ground-truth circles. A single community is considered perfectly homogenous if its members contain the same members as a given circle. Extending this thought to the entire network, a homogenous network is one with its communities closely resembling the actual circles. Conversely, an inhomogenous grouping means communities contain nodes from many different ground-truth circles.

Completeness is a measure of how similar the circles are to the communities partitioned by an algorithm. A circle is considered perfectly complete if all its members are placed in the same community by the algorithm; conversely, a circle with a low completeness value means its members were split up into different communities. As such, a network with high completeness means the circles were preserved well across all the communities.

Question 22

```
In [30]: def gplus_circles():
    """
    Reads the directory gplus/ files and reads in the set of circles for each ego node.
    """
    # Find all .circles files
    current_dir = path.join(os.getcwd(), 'gplus')
    circles_files = glob.glob(os.path.join(current_dir, "*.circles"))

    # Iterate through each user
    network_circles = {}
    for circles_fpath in circles_files:
        # Extract nodeId from filename (e.g., "12345.circles" -> "12345")
        filename = path.basename(circles_fpath)
        node_id = filename.split(".")[0] # Ego node

        # Parse circles for this ego node
        circles = {}
        with open(circles_fpath, "r") as f:
            for line in f.readlines():
                tokens = line.strip().split()
                if not tokens: continue
                circle_id = tokens[0]
                node_ids = [str(tok) for tok in tokens[1:]]
                circles[circle_id] = node_ids

        # Cache circles for nodeId
        network_circles[node_id] = circles
```

```
return network_circles
```

```
In [62]: import numpy as np
```

```
def compute_homogeneity_and_completeness(node_id, g, communities, circles, all_nodes):
    """
    Computes the homogeneity and completeness scores for a given network `g`
    and accompanying `user_circles`.
    """
    N = len(all_nodes)
    if N == 0:
        print('WARNING: This graph has zero nodes!')
        return 0, 0

    C_list = list(circles.values())
    K_list = communities

    num_circles = len(C_list)    # |C|
    num_communities = len(K_list) # |K|
    if num_circles == 0 or num_communities == 0:
        print(f'WARNING: num_circles={num_circles}, num_communities={num_communities}')
        return 0, 0

    # Matrix A: number of people belonging to community j and circle i
    A = np.zeros((num_communities, num_circles))
    for j, community in enumerate(K_list):
        # Cast all IDs to str to avoid set mismatch bugs
        # Extract IDs and *exclude* the main ego node_id (accounting for .circles format)
        set_K = {
            str(g.vs[node_idx]['snap_id'])
            for node_idx in community
            if str(g.vs[node_idx]['snap_id']) != str(node_id)
        }
        for i, circle in enumerate(C_list):
            set_C = set(str(circ) for circ in circle)
            A[j, i] = len(set_K.intersection(set_C))

    # Calculate N (total number of people with circle information)
    N = np.sum(A)
    if N == 0:
        print('WARNING: Zero assignments')
        return 0, 0

    # Entropies H(C) and H(K)
    a = np.sum(A, axis=0) # Number of people in circle C_i
    b = np.sum(A, axis=1) # Number of people in community K_j
    def entropy(values):
        values_ = values[values > 0]
        return -np.sum(values_ * np.log(values_))
    H_C = entropy(a / N)
    H_K = entropy(b / N)

    # Conditional entropies
    H_C_K = 0.0
    for j in range(num_communities):
        if b[j] > 0:
            for i in range(num_circles):
                if A[j, i] > 0:
                    H_C_K += (A[j, i] / N) * np.log(A[j, i] / b[j])
    H_C_K *= -1

    H_K_C = 0.0
    for i in range(num_circles):
        if a[i] > 0:
            for j in range(num_communities):
                if A[j, i] > 0:
                    H_K_C += (A[j, i] / N) * np.log(A[j, i] / a[i])
    H_K_C *= -1

    # Computing h and c
    h = 1.0 - (H_C_K / H_C) if H_C > 0 else 1.0
    c = 1.0 - (H_K_C / H_K) if H_K > 0 else 1.0

    return h, c
```

```
In [63]: def characterize_networks(node_id, gplus_networks, networks_circles):
    """
    Characterizes the network for a given node ID by computing the homogeneity
    and completeness values.
    """
    # Get network and circles for user_id
    g = gplus_networks[node_id]
    user_circles = networks_circles[node_id]

    # Obtain communities
    dendrogram = g.community_walktrap()
    communities = dendrogram.as_clustering()
    n_communities = len(communities)

    # Compute homogeneity and completeness
    all_nodes = g.vs['snap_id']
```

```
h, c = compute_homogeneity_and_completeness(node_id, g, communities, user_circles, all_nodes)
print(f"(Node ID: {node_id}) h={h:.4f}, c={c:.4f}")
```

```
In [64]: # Compute h and c
specific_ids = ['109327480479767108490', '115625564993990145546', '101373961279443806744']
network_circles = gplus_circles()
for node_id in specific_ids:
    characterize_networks(node_id, gplus_networks, network_circles)
```

```
(Node ID: 109327480479767108490) h=0.5249, c=0.5497
```

```
(Node ID: 115625564993990145546) h=0.1060, c=0.3822
```

```
(Node ID: 101373961279443806744) h=0.0004, c=0.0009
```

Starting with the worst performance, network 101373961279443806744 has very low h and c-values, indicating that the algorithm essentially failed to detect the right communities.

Conversely, the best performance is shown by network 109327480479767108490. The $h \approx 0.5$ indicates about half the nodes in each partitioned community actually belong to the ground-truth circles. The $c \approx 0.5$ indicates about half of the members in the ground-truth circles were successfully assigned to the same community. This is fairly decent performance.

Network 115625564993990145546 has a low h value, meaning communities are composed of nodes/members who are actually from many different circles. The c value is moderate at ~ 0.4 , meaning the members from each circle were somewhat kept together in the same community. Low homogeneity with moderate completeness means that while members from the same circles were grouped together in the same community, a low number of communities were formed by the algorithm to correctly split the circle members to their respective separate communities.

3. Cora Dataset

```
In [7]: import random
import numpy as np
import networkx as nx
import torch
import torch.nn.functional as F
from torch_geometric.datasets import Planetoid
from torch_geometric.nn import GCNConv
from torch_geometric.utils import to_networkx
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score

import matplotlib.pyplot as plt
from sklearn.manifold import TSNE

def load_cora_dataset(root_dir='./data/Cora'):
    print("Loading Cora Dataset...")
    dataset = Planetoid(root=root_dir, name='Cora')
    return dataset, dataset[0]

def set_seed(seed=232):
    """Locks random seeds across Python, NumPy, and PyTorch"""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
        torch.backends.cudnn.deterministic = True
        torch.backends.cudnn.benchmark = False

dataset, data = load_cora_dataset()
```

Loading Cora Dataset...

Question 23

```
In [8]: def plot_gcn_tsne(out, labels):
    tsne = TSNE(n_components=2, random_state=232)
    z = tsne.fit_transform(out.detach().cpu().numpy())

    plt.figure(figsize=(8, 6))
    scatter = plt.scatter(z[:, 0], z[:, 1], c=labels.cpu().numpy(), cmap='tab10', s=15, alpha=0.8)
    plt.colorbar(scatter, label='Class Label')
    plt.title('Idea 1: GCN Node Representations (2D t-SNE Projection)', fontsize=12, fontweight='bold')
    plt.xlabel('t-SNE Dimension 1')
    plt.ylabel('t-SNE Dimension 2')
    plt.grid(True, linestyle='--', alpha=0.3)
    plt.tight_layout()
    plt.show()

def run_idea_1(data, num_features, num_classes, hidden_dim=16, lr=0.01, weight_decay=5e-4, epochs=200):
    print("\nIDEA 1:")

    set_seed(232)
    X, y, edge_index = data.x, data.y, data.edge_index
    train_mask, test_mask = data.train_mask, data.test_mask

    model = GCN(num_features, hidden_dim, num_classes)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr, weight_decay=weight_decay)

    # Training
    model.train()
    for _ in range(epochs):
        optimizer.zero_grad()
        out = model(X, edge_index)
        loss = F.nll_loss(out[train_mask], y[train_mask])
        loss.backward()
        optimizer.step()

    # Evaluation
    model.eval()
    with torch.no_grad():
        out = model(X, edge_index)
        _, pred = out.max(dim=1)
        correct = float(pred[test_mask].eq(y[test_mask]).sum().item())
        gcn_acc = correct / test_mask.sum().item()

    print(f"GCN Test Accuracy: {gcn_acc * 100:.2f}%")

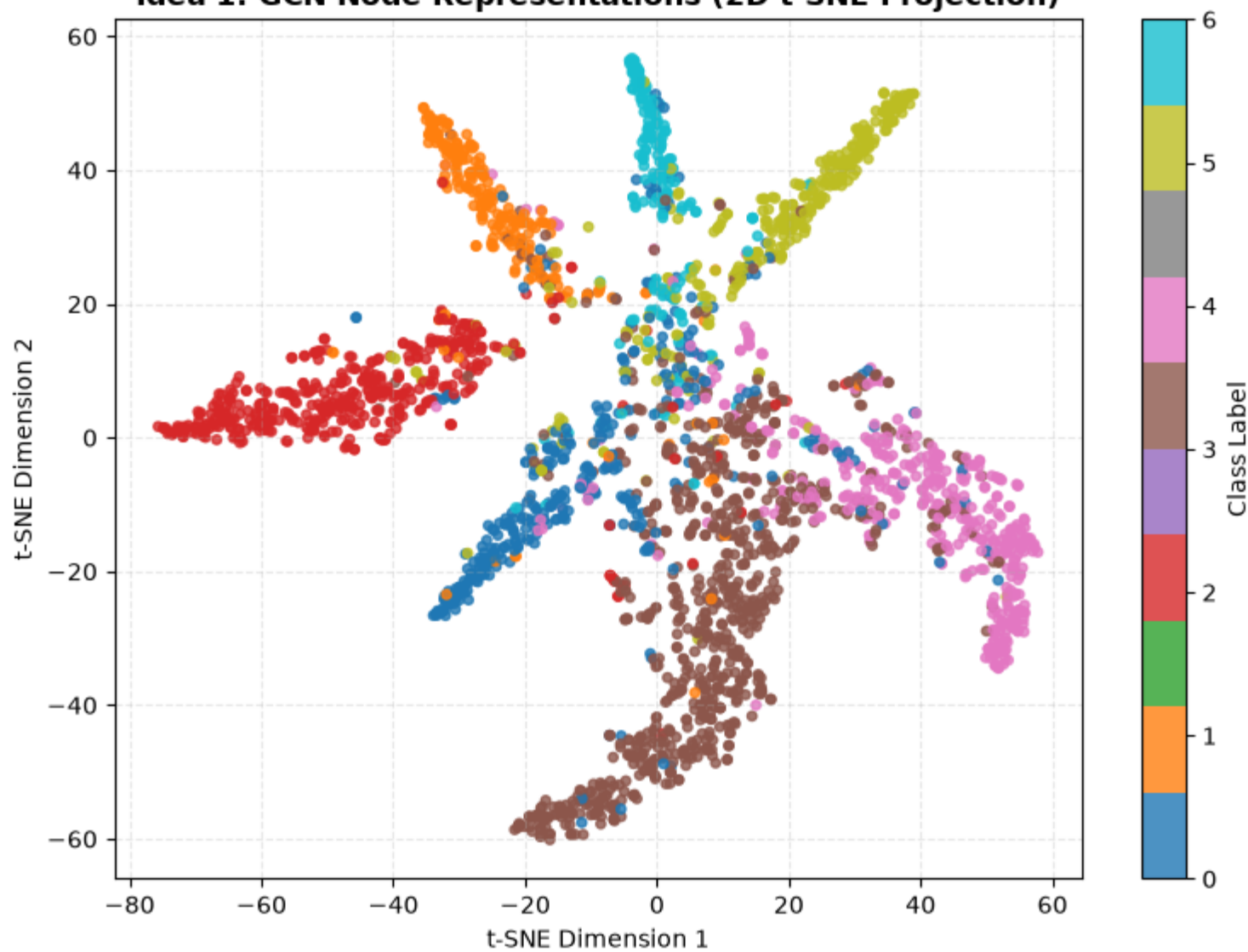
    # Return out and y so they are available outside the function
    return gcn_acc, out, y

# Run training and plot
gcn_acc, out, y = run_idea_1(data, dataset.num_node_features, dataset.num_classes)
plot_gcn_tsne(out, y)
```

IDEA 1:

GCN Test Accuracy: 80.70%

Idea 1: GCN Node Representations (2D t-SNE Projection)



Question 23 Discussion

- Number of Layers: A 2-layer GCN architecture, which allows each node to aggregate feature information from its 2-hop neighborhood. Going beyond on layers on Cora can lead to oversmoothing, causing node representations to collapse and become indistinguishable across classes.
- Hyperparameters:
 - Hidden Dimensions: 16
 - Learning Rate: 0.01
 - Weight Decay (L_2 regularization): 5×10^{-4}
 - Dropout Rate: 0.5
 - Training Epochs: 200
 - Optimizer: Adam
- Performance: The model achieves a test accuracy of 80.70% for the 140-seed split, so jointly learning node text features alongside local graph topology works well.

Question 24

```
In [9]: class PyTorchNode2Vec(torch.nn.Module):
    def __init__(self, num_nodes, embedding_dim):
        super().__init__()
        self.in_embed = torch.nn.Embedding(num_nodes, embedding_dim)
        self.out_embed = torch.nn.Embedding(num_nodes, embedding_dim)
        self.in_embed.weight.data.uniform_(-0.5 / embedding_dim, 0.5 / embedding_dim)
        self.out_embed.weight.data.uniform_(-0.5 / embedding_dim, 0.5 / embedding_dim)

    def forward(self, tgt, ctx, neg):
        tgt_emb = self.in_embed(tgt)
        ctx_emb = self.out_embed(ctx)
        pos_loss = torch.sigmoid(torch.sum(tgt_emb * ctx_emb, dim=-1))
        pos_loss = -torch.log(pos_loss + 1e-15).mean()

        neg_emb = self.out_embed(neg)
        neg_loss = torch.sigmoid(-torch.sum(neg_emb * tgt_emb.unsqueeze(1), dim=-1))
        neg_loss = -torch.log(neg_loss + 1e-15).sum(dim=-1).mean()

        return pos_loss + neg_loss

def train_node2vec_embeddings(data, embedding_dim=64, walk_length=20, walks_per_node=10,
                             window_size=5, num_epochs=5, batch_size=4096, lr=0.01):
    """Generates random walks and trains Skip-Gram Node2Vec embeddings utilizing PyTorch."""
    print("Generating random walks...")
    G = to_networkx(data, to_undirected=True)
    nodes = list(G.nodes())
    walks = []

    for _ in range(walks_per_node):
        random.shuffle(nodes)
        for node in nodes:
            walk = [node]
            while len(walk) < walk_length:
                cur = walk[-1]
                neighbors = list(G.neighbors(cur))
```

```

        if not neighbors:
            break
        walk.append(random.choice(neighbors))
        walks.append(walk)

print("Creating training pairs...")
pairs = []
for walk in walks:
    for i, target in enumerate(walk):
        start = max(0, i - window_size)
        end = min(len(walk), i + window_size + 1)
        for j in range(start, end):
            if i != j:
                pairs.append((target, walk[j]))

pairs = np.array(pairs)
targets = torch.tensor(pairs[:, 0], dtype=torch.long)
contexts = torch.tensor(pairs[:, 1], dtype=torch.long)

print("Training Node2Vec embeddings via PyTorch...")
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
num_nodes = data.num_nodes

n2v_net = PyTorchNode2Vec(num_nodes, embedding_dim).to(device)
optimizer = torch.optim.Adam(n2v_net.parameters(), lr=lr)

n2v_net.train()
for epoch in range(num_epochs):
    permutation = torch.randperm(targets.size(0))
    epoch_loss = 0
    for i in range(0, targets.size(0), batch_size):
        indices = permutation[i:i + batch_size]
        batch_tgt = targets[indices].to(device)
        batch_ctx = contexts[indices].to(device)
        batch_neg = torch.randint(0, num_nodes, (batch_tgt.size(0), 5), device=device)

        optimizer.zero_grad()
        loss = n2v_net(batch_tgt, batch_ctx, batch_neg)
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item()
    print(f"Epoch {epoch+1}/{num_epochs} | Loss: {epoch_loss / (targets.size(0)/batch_size):.4f}")

n2v_net.eval()
with torch.no_grad():
    node_embeddings = n2v_net.in_embed.weight.cpu().numpy()

return node_embeddings

def run_idea_2(data):
    print("\nIDEA 2:")
    set_seed(232)

    # Generate structural embeddings
    node_embeddings = train_node2vec_embeddings(data)

    # Setup features and masks
    text_features = data.x.numpy()
    labels = data.y.numpy()

    train_idx = data.train_mask.nonzero(as_tuple=False).view(-1).numpy()
    test_idx = data.test_mask.nonzero(as_tuple=False).view(-1).numpy()

    # Initialize Classifiers
    rf_n2v = RandomForestClassifier(n_estimators=100, random_state=42)
    rf_text = RandomForestClassifier(n_estimators=100, random_state=42)
    rf_combined = RandomForestClassifier(n_estimators=100, random_state=42)

    print("\nEvaluating Classifiers")
    # Run A: Node2Vec Only
    rf_n2v.fit(node_embeddings[train_idx], labels[train_idx])
    preds_n2v = rf_n2v.predict(node_embeddings[test_idx])
    print(f"Node2Vec Only Accuracy: {accuracy_score(labels[test_idx], preds_n2v) * 100:.2f}%")

    # Run B: Text Features Only
    rf_text.fit(text_features[train_idx], labels[train_idx])
    preds_text = rf_text.predict(text_features[test_idx])
    print(f"Text Features Only Accuracy: {accuracy_score(labels[test_idx], preds_text) * 100:.2f}%")

    # Run C: Combined Features
    combined_features = np.hstack([node_embeddings, text_features])
    rf_combined.fit(combined_features[train_idx], labels[train_idx])
    preds_combined = rf_combined.predict(combined_features[test_idx])
    print(f"Combined Features Accuracy: {accuracy_score(labels[test_idx], preds_combined) * 100:.2f}%")

run_idea_2(data)

```

```
IDEA 2:
Generating random walks...
Creating training pairs...
Training Node2Vec embeddings via PyTorch...
Epoch 1/5 | Loss: 0.7436
Epoch 2/5 | Loss: 0.5547
Epoch 3/5 | Loss: 0.5487
Epoch 4/5 | Loss: 0.5462
Epoch 5/5 | Loss: 0.5436
```

```
Evaluating Classifiers
Node2Vec Only Accuracy: 64.20%
Text Features Only Accuracy: 56.30%
Combined Features Accuracy: 69.80%
```

Question 24 Discussion

Node2Vec generation process includes simulating second-order random walks parameterized by a return parameter p and an in-out parameter q to trade off between local community exploration (BFS) and macro-structure discovery (DFS). Then apply Skip-Gram Architecture by treating these random walk paths as sentences and node IDs as words, with negative sampling to maximize the log-probability of predicting a node's network neighborhood context given its embedding.

Node2Vec alone outperforms the 1433-dimensional text features alone by 7.9%. Standard random forest classifiers struggle with high-dimensional, highly sparse feature spaces (1433 binary features where most values are 0) making individual decision tree splits on single sparse words noisy and weak. Node2Vec condenses complex graph connectivity into a dense, low-dimensional (64-dim) continuous manifold, allowing random forest decision boundaries to separate paper topics much more effectively. Combining structural embeddings with content features (Node2Vec || Text) achieves the overall highest classification accuracy of 69.8%, confirming that graph topology and text content provide complementary signals.

Question 25

```
In [10]: def precompute_transition_probs(G_gcc, X):
        """Calculates custom edge transition probabilities using softmax over feature dot products."""
        print("Computing custom transition probabilities...")
        transition_probs = {}
        for u in G_gcc.nodes():
            neighbors = list(G_gcc.neighbors(u))
            if not neighbors:
                continue

            u_feat = X[u]
            neigh_feats = X[neighbors]

            dot_products = torch.matmul(neigh_feats, u_feat)
            exp_vals = torch.exp(dot_products)
            probs = exp_vals / torch.sum(exp_vals)

            transition_probs[u] = {n: p.item() for n, p in zip(neighbors, probs)}

        return transition_probs

def run_idea_3(dataset, data, walks_per_seed=1000, max_steps=15, teleport_probs=(0, 0.1, 0.2)):
    print("\nIDEA 3: Customized Personalized PageRank")

    set_seed(232)
    # 1. Extract Giant Connected Component
    G = to_networkx(data, to_undirected=True)
    gcc_nodes = max(nx.connected_components(G), key=len)
    G_gcc = G.subgraph(gcc_nodes).copy()
    print(f"GCC extracted with {G_gcc.number_of_nodes()} nodes.")

    # 2. Precompute transition probabilities
    transition_probs = precompute_transition_probs(G_gcc, data.x)

    # 3. Prepare Seed Documents and Unlabeled Nodes within GCC
    classes = range(dataset.num_classes)
    seed_docs = {c: [] for c in classes}
    unlabeled_nodes = []

    for node in G_gcc.nodes():
        if data.train_mask[node]:
            seed_docs[data.y[node].item()].append(node)
        elif data.test_mask[node]:
            unlabeled_nodes.append(node)

    for c in classes:
        if len(seed_docs[c]) == 0:
            print(f"Warning: No seed documents found in GCC for class {c}")

    # 4. Random Walk Simulation for each Teleportation Probability
    for alpha in teleport_probs:
        print(f"\nRunning PPR with Teleportation Probability = {alpha}")
        visit_counts = {node: np.zeros(dataset.num_classes) for node in unlabeled_nodes}

        for c in classes:
            seeds = seed_docs[c]
            if not seeds:
                continue
```



```

        for start_node in seeds:
            for _ in range(walks_per_seed):
                curr_node = start_node

                for step in range(max_steps):
                    if random.random() < alpha:
                        curr_node = random.choice(seeds)
                    else:
                        neighbors = list(G_gcc.neighbors(curr_node))
                        if not neighbors:
                            break

                        probs = [transition_probs[curr_node][n] for n in neighbors]
                        curr_node = random.choices(neighbors, weights=probs, k=1)[0]

                    if curr_node in visit_counts:
                        visit_counts[curr_node][c] += 1

# 5. Prediction and Evaluation
y_true = []
y_pred = []

for node in unlabeled_nodes:
    counts = visit_counts[node]
    if counts.sum() == 0:
        pred_class = random.choice(list(classes))
    else:
        pred_class = np.argmax(counts)

    y_pred.append(pred_class)
    y_true.append(data.y[node].item())

acc = accuracy_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred, average='macro')

print(f"Accuracy: {acc * 100:.2f}% | F1 Score (Macro): {f1:.4f}")

run_idea_3(dataset, data)
```

IDEA 3: Customized Personalized PageRank
GCC extracted with 2485 nodes.
Computing custom transition probabilities...

Running PPR with Teleportation Probability = 0
Accuracy: 64.81% | F1 Score (Macro): 0.6392

Running PPR with Teleportation Probability = 0.1
Accuracy: 63.50% | F1 Score (Macro): 0.6276

Running PPR with Teleportation Probability = 0.2
Accuracy: 63.39% | F1 Score (Macro): 0.6250

Question 25 Discussion

Performance (GCC: 2,485 nodes)

Teleportation Prob (α)	Test Accuracy	Macro F1 Score
0.0	64.81%	0.6392
0.1	63.50%	0.6276
0.2	63.39%	0.6250

Setting $\alpha = 0$ yielded the highest accuracy (64.81%) and macro F1 score (0.6392). Because our random walk transitions are heavily guided by content similarity, allowing walks to run purely along high-similarity edges keeps the walker within localized and relevant topic clusters. $\alpha = 0.1, 0.2$ lets walker teleport back to any seed node of that class. If seed nodes lie in distinct or loosely connected sub-communities of the same topic, teleporting can introduce noise into local visit distributions for boundary nodes thus slightly reducing classification accuracy.